

INSTITUTO TECNOLÓGICO DE AERONÁUTICA



Leonardo Lins

**SEPARAÇÃO ENTRE CANAL DE INSTRUÇÃO E CANAL
DE DADO EM GUARDRAILS DE ENTRADA PARA
COMPONENTES DE LLM**

Trabalho de Graduação
2026

Curso de Engenharia da Computação

Leonardo Lins

**SEPARAÇÃO ENTRE CANAL DE INSTRUÇÃO E
CANAL DE DADO EM GUARDRAILS DE
ENTRADA PARA COMPONENTES DE LLM**

Orientador

Prof. Dr. Cesar Augusto Cavalheiro Marcondes (ITA)

ENGENHARIA DA COMPUTAÇÃO

**SÃO JOSÉ DOS CAMPOS
INSTITUTO TECNOLÓGICO DE AERONÁUTICA**

Dados Internacionais de Catalogação-na-Publicação (CIP)
Divisão de Informação e Documentação

Lins, Leonardo

Separação entre Canal de Instrução e Canal de Dado em Guardrails de Entrada para Componentes de LLM / Leonardo Lins.

São José dos Campos, 2026.

55f.

Trabalho de Graduação – Curso de Engenharia da Computação– Instituto Tecnológico de Aeronáutica, 2026. Orientador: Prof. Dr. Cesar Augusto Cavalheiro Marcondes.

1. Segurança de computadores. 2. Processamento de linguagem natural. 3. Aprendizado de máquina. I. Instituto Tecnológico de Aeronáutica. II. Título.

REFERÊNCIA BIBLIOGRÁFICA

LINS, Leonardo. **Separação entre Canal de Instrução e Canal de Dado em Guardrails de Entrada para Componentes de LLM**. 2026. 55f. Trabalho de Conclusão de Curso (Graduação) – Instituto Tecnológico de Aeronáutica, São José dos Campos.

CESSÃO DE DIREITOS

NOME DO AUTOR: Leonardo Lins

TÍTULO DO TRABALHO: Separação entre Canal de Instrução e Canal de Dado em Guardrails de Entrada para Componentes de LLM.

TIPO DO TRABALHO/ANO: Trabalho de Conclusão de Curso (Graduação) / 2026

É concedida ao Instituto Tecnológico de Aeronáutica permissão para reproduzir cópias deste trabalho de graduação e para emprestar ou vender cópias somente para propósitos acadêmicos e científicos. O autor reserva outros direitos de publicação e nenhuma parte deste trabalho de graduação pode ser reproduzida sem a autorização do autor.

Leonardo Lins

ENDEREÇO DO AUTOR

CEP – São José dos Campos–SP

SEPARAÇÃO ENTRE CANAL DE INSTRUÇÃO E CANAL DE DADO EM GUARDRAILS DE ENTRADA PARA COMPONENTES DE LLM

Essa publicação foi aceita como Relatório Final de Trabalho de Graduação

Leonardo Lins

Autor

Cesar Augusto Cavalheiro Marcondes (ITA)

Orientador

Prof. Dr. COORDENADOR DO CURSO

Coordenador do Curso de Engenharia da Computação

São José dos Campos, DD de mês de 2026.

A quem me ensinou que desconfiar de
um texto é uma forma de respeito por
ele.

Agradecimentos

Ao meu orientador, Prof. Dr. Cesar Augusto Cavalheiro Marcondes, pela leitura atenta e pela insistência em que o resultado fosse medido onde ele de fato importa.

Aos professores e colegas do curso, pelas conversas que mudaram o rumo deste trabalho mais de uma vez.

À minha família, pelo apoio ao longo de toda a graduação.

“Um programa que não distingue o que lhe mandam do que lhe contam está condenado a obedecer a estranhos.”

AUTOR DA EPÍGRAFE

Resumo

O uso de LLMs como componentes fixos de software expõe uma superfície de ataque linguística cuja causa-raiz é a indistinção nativa entre inputs maliciosos e não maliciosos sobre a entrada do modelo, da qual decorre o *prompt injection*. A defesa, consolidada cientificamente e comercialmente em implementações industriais, interpõe um *guardrail* de entrada antes da execução principal. Esse filtro, porém, é também uma LLM que recebe instrução e dado pelo mesmo canal, e herda a vulnerabilidade que existe para mitigar. Este trabalho mede o que se ganha ao aplicar ao próprio filtro a separação entre canal de instrução e canal de dado que a literatura propõe para o modelo principal, e ao fazer com que o filtro preserve e marque o trecho suspeito em vez de removê-lo, repassando a anotação ao componente que executará a tarefa. Essencialmente, o objetivo é melhorar a segurança de sistemas que utilizem LLM sem recorrer a um modelo *fine-tunado*, mas adicionando técnicas de segurança que modificam o *input*. O desenho experimental é 2×2 : delimitação em XML na entrada do filtro e marcação seletiva na sua saída, totalizando quatro condições, além de um teste sem filtro como referência inicial e de um filtro *fine-tunado* da mesma família, avaliado nas mesmas quatro condições, como ponto de comparação. A avaliação emprega dois modelos abertos sem *fine-tuning*, cada um instanciando o sistema inteiro, sobre três *benchmarks* públicos: BIPIA, SEP e NotInject. São 42 execuções por repetição, repetidas dez vezes: 420 execuções e 131.460 registros. O desfecho de cada item é a média das dez repetições, medido na saída do componente principal, em taxa de sucesso do ataque, utilidade em tarefas benignas e sobre-bloqueio, com intervalos de confiança por *bootstrap* pareado sobre os mesmos itens. A combinação dos dois fatores reduziu a taxa de sucesso do ataque no BIPIA em 0,084 para o Llama-3.1-8B e em 0,139 para o gpt-oss-20b, ambas com intervalo de 95% afastado do zero. Sobre o filtro *fine-tunado* a mesma combinação reduziu 0,139, o que mostra que o ganho não depende de o filtro ser um modelo de uso geral. Marcar em vez de remover mostrou-se uma alternativa viável às defesas que removem, com proteção maior que a do modelo *fine-tunado* da mesma família e com facilidade de implementação muito maior.

Palavras-Chave: *Prompt injection*. *Guardrails* de entrada. Separação entre instrução e dado. *Spotlighting*. Segurança de LLM. Modelos abertos.

Abstract

Using LLMs as fixed software components exposes a linguistic attack surface whose root cause is the native inability to distinguish malicious from non-malicious inputs at the model’s input, from which *prompt injection* follows. The defense, consolidated both scientifically and commercially in industrial deployments, places an input *guardrail* before the main execution. That filter, however, is itself an LLM that receives instruction and data through the same channel, and inherits the very vulnerability it exists to mitigate. This work measures what is gained by applying to the filter itself the separation between instruction channel and data channel that the literature proposes for the main model, and by having the filter preserve and mark the suspicious span instead of removing it, forwarding the annotation to the component that will carry out the task. In essence, the goal is to improve the security of LLM-based systems without resorting to a *fine-tuned* model, but by adding security techniques that modify the *input*. The experimental design is 2×2 : XML delimiting at the filter’s input and selective marking at its output, totaling four conditions, plus a no-filter test as the initial reference and a *fine-tuned* filter from the same family, evaluated under the same four conditions, as a point of comparison. The evaluation uses two open models without *fine-tuning*, each one instantiating the whole system, over three public *benchmarks*: BIPIA, SEP and NotInject. There are 42 runs per repetition, repeated ten times: 420 runs and 131,460 records. The outcome of each item is its mean over the ten repetitions, measured at the output of the main component, in attack success rate, utility on benign tasks and over-blocking, with paired *bootstrap* confidence intervals over the same items. Combining both factors reduced the attack success rate on BIPIA by 0.084 for Llama-3.1-8B and by 0.139 for gpt-oss-20b, both with a 95% interval away from zero. On the *fine-tuned* filter the same combination reduced it by 0.139, showing that the gain does not depend on the filter being a general-purpose model. Marking rather than removing proved a viable alternative to removal-based defenses, with greater protection than the *fine-tuned* model from the same family and far easier implementation.

Keywords: Prompt injection. Input guardrails. Instruction–data separation. Spotligh-
ting. LLM security. Open models.

Lista de Figuras

- FIGURA 3.1 – Fluxo da informação. O filtro recebe instrução e dado por canais separados; ao liberar a requisição, marca os trechos suspeitos e os repassa anotados ao componente principal. A medição ocorre na saída do sistema, não na classificação do filtro. 24
- FIGURA 4.1 – Taxa de sucesso do ataque por configuração. O painel do NotInject fica vazio porque o conjunto não tem itens atacados. 33
- FIGURA 4.2 – Diferença na taxa de sucesso do ataque de cada condição em relação ao *prompt* simples. As hastes são intervalos de confiança de 95% por *bootstrap* BCa pareado sobre os itens. 34
- FIGURA 4.3 – Efeitos do desenho 2×2 sobre a taxa de sucesso do ataque. O efeito principal de um fator é a média das duas condições com o fator menos a média das duas sem ele; a interação é a diferença entre o ganho da marcação com e sem delimitação. As hastes são intervalos de confiança de 95% por *bootstrap* BCa sobre os itens. 35
- FIGURA 4.4 – Diferença de utilidade de cada condição em relação ao *prompt* simples, com intervalos de confiança de 95% por *bootstrap* BCa pareado sobre os itens. Só o BIPIA publica resposta de referência; no SEP e no NotInject, a utilidade significa “respondeu sem ter sido bloqueado”, e não “respondeu corretamente”. 36
- FIGURA 4.5 – Efeitos do desenho 2×2 sobre a utilidade, definidos como na Figura 4.3, com intervalos de confiança de 95% por *bootstrap* BCa sobre os itens. Vale a ressalva da Figura 4.4 sobre o significado da utilidade. 37

- FIGURA 4.6 – Utilidade: fração das tarefas legítimas cumpridas, com item bloqueado contando como fracasso. As duas barras sem haste, do Llama-3.1-8B sem filtro no SEP e no NotInject, valem exatamente 1,000, sem variação para um intervalo. Vale a ressalva da Figura 4.4 sobre o significado da utilidade. 38
- FIGURA 4.7 – Sobre-bloqueio: fração dos itens benignos que o filtro bloqueou. É a única métrica definida nos três benchmarks e o motivo de incluir o NotInject, composto de requisições legítimas com vocabulário sensível. Sem filtro não há decisão de bloqueio, e a barra correspondente é nula por construção. As barras sem haste do gpt-oss-20b e do gpt-oss-safeguard no BIPIA e no SEP somam de zero a quatro bloqueios em 1.500 decisões, poucos demais para um intervalo. 39
- FIGURA 4.8 – Alcance da marcação seletiva: fração dos itens liberados pelo filtro com ao menos um trecho marcado. É zero por construção nas condições que não marcam. 40
- FIGURA 4.9 – Taxa de saída inválida do filtro: fração das respostas que não seguiram o contrato de saída. Saída inválida conta como não bloqueio: o item segue para o componente principal e entra na medição. 41
- FIGURA 4.10 – Resposta vazia do componente principal: fração dos itens liberados pelo filtro em que o componente não produziu resposta, quase sempre por esgotar o teto de 8.192 *tokens* raciocinando. Resposta vazia conta como saída inválida, e não como bloqueio nem como sobre-bloqueio: não executa o ataque e não cumpre a tarefa. 42
- FIGURA 4.11 – *Tokens* por item, somando os dois estágios. A diferença para o *prompt* simples é a despesa da delimitação e da marcação, que acrescentam texto; a diferença para a configuração sem filtro é a despesa do filtro, que dobra o número de chamadas. 43
- FIGURA 4.12 – Tempo de geração por item, em segundos, somando os dois estágios. A barra é a mediana e o traço, o percentil 95. O tempo é o informado pelo provedor para cada chamada, e por isso não inclui fila, rede nem espera por limite de taxa. O mesmo modelo é servido por provedores com hardware diferente, escolhidos a cada chamada pela interface, e essa variação entra no tempo medido. 44

Lista de Tabelas

TABELA 3.1 – As quatro condições do desenho fatorial.	26
TABELA 4.1 – Métricas por condição e modelo no benchmark BIPIA, médias de 10 repetições.	45
TABELA 4.2 – Métricas por condição e modelo no benchmark SEP, médias de 10 repetições.	45
TABELA 4.3 – Métricas por condição e modelo no benchmark NotInject, médias de 10 repetições.	46

Lista de Abreviaturas e Siglas

API	<i>application programming interface</i>
ASR	<i>attack success rate</i> , taxa de sucesso do ataque
BIPIA	<i>Benchmark for Indirect Prompt Injection Attacks</i>
CIP	catalogação na publicação
FN	falso negativo
FP	falso positivo
FPR	<i>false positive rate</i> , taxa de falsos positivos
IC	intervalo de confiança
IPI	<i>indirect prompt injection</i> , injeção indireta de <i>prompt</i>
JSON	<i>JavaScript Object Notation</i>
LLM	<i>large language model</i> , modelo de linguagem de grande porte
LLMC	<i>LLM component</i> , LLM empregada como componente de software
MoE	<i>mixture of experts</i> , mistura de especialistas
RAG	<i>retrieval-augmented generation</i>
RLHF	<i>reinforcement learning from human feedback</i>
SEP	<i>Should It Be Executed or Processed?</i>
TPR	<i>true positive rate</i> , taxa de verdadeiros positivos
XML	<i>eXtensible Markup Language</i>

Lista de Símbolos

$\hat{}$	marca aposta pelo filtro sobre trecho julgado perigoso
<code><data></code>	tag que delimita o conteúdo não confiável
Δ	diferença de uma métrica entre uma condição e sua referência
n	número de itens executados em uma configuração
p	taxa estimada sobre os itens de uma configuração

Sumário

1	INTRODUÇÃO	16
1.1	Objetivo	17
2	FUNDAMENTAÇÃO TEÓRICA	19
2.1	A LLM como componente e a interface do filtro	19
2.2	<i>Prompt injection</i> : direto e indireto	19
2.3	A não separação como propriedade mensurável	20
2.4	<i>Guardrails</i> de entrada e sua circularidade	20
2.5	<i>Spotlighting</i> : separação por convenção	21
2.6	Filtros que sanitizam a entrada	22
2.7	Métricas de avaliação	22
3	METODOLOGIA	24
3.1	Fluxo da informação	24
3.2	Primeiro fator: delimitação na entrada do filtro	24
3.3	Segundo fator: marcação seletiva na saída do filtro	25
3.4	Linhas: modelos e referências	26
3.5	Benchmarks	27
3.6	Métricas e protocolo	27
3.7	Situando o resultado frente ao <i>fine-tuning</i>	30
3.8	Execução e consolidação	30
3.8.1	Orçamento de geração	31
4	RESULTADOS	32

4.1	Taxa de sucesso do ataque	32
4.2	Custo da defesa	35
4.3	Alcance da marcação seletiva	39
4.4	Saídas inválidas	40
4.5	Custo operacional	42
4.6	Tabelas completas	44
5	CONCLUSÃO	47
5.1	O ganho da separação de canal	47
5.2	A dimensão do efeito	48
5.3	O custo	48
5.4	O teto do método	49
5.5	Comparação com o filtro treinado	50
5.6	Limitações	51
	REFERÊNCIAS	53

1 Introdução

Os avanços recentes nas LLMs deslocaram o seu papel de meros assistentes de geração de texto para o de componentes fixos dentro do software. Em vez de depender exclusivamente de código pré-estabelecido para interpretar a intenção do usuário e acionar a funcionalidade correspondente, o sistema passa a delegar parte dessa interpretação a uma LLM. Essa mudança traz ganhos de flexibilidade, mas introduz uma classe de preocupações de segurança sem equivalente direto na arquitetura tradicional de software.

A raiz dessas preocupações é estrutural. Na arquitetura *Transformer* (VASWANI *et al.*, 2017), instruções do administrador e a entrada do usuário são processados pelo mesmo canal de entrada, sem uma separação sintática garantida pelo modelo. Desta maneira, ordens provenientes do usuário não confiável são recebidas com a mesma autoridade que o modelo confere às instruções do administrador. De forma diferente de um banco de dados, em que um analisador determinístico distingue a estrutura da consulta de seus valores, na LLM qualquer conteúdo textual que entre no contexto pode ser interpretado como instrução se for formulado dessa forma. É dessa indistinção que nasce o *prompt injection*, demonstrado de forma sistemática já em 2022 (PEREZ; RIBEIRO, 2022) e posteriormente formalizado como problema padronizado de avaliação (LIU *et al.*, 2024). O caso mais perigoso é a injeção indireta, na qual a instrução maliciosa não é digitada pelo atacante, mas plantada em conteúdo externo que o componente processa no curso normal de suas operações (GRESHAKE *et al.*, 2023).

Como a superfície de ataque inclui a totalidade do espaço linguístico que pode alcançar o modelo, e não apenas o que o usuário digita, a prática consolidada é interpor, antes da execução principal, um *guardrail* de entrada (DONG *et al.*, 2024): uma camada que classifica, normaliza ou bloqueia a requisição antes que ela alcance os demais módulos. Quando a decisão é tomada por um modelo treinado para essa função, tem-se um *guard model* (INAN *et al.*, 2023; ZENG *et al.*, 2024), hoje a abordagem dominante e disponível em famílias abertas (ZHAO *et al.*, 2025) e em sistemas de produção (CHENNABASAPPA *et al.*, 2025). Via de regra, modelos usados como *guard* são mais leves e conseqüentemente mais baratos de operar do que o modelo principal, visto que sua função é mais específica e limitada, ao mesmo tempo que os desenvolvedores buscam não escalar o custo de operação do sistema como um todo.

Há aqui uma circularidade pouco explorada. O filtro que deve proteger o componente principal contra instruções embutidas em dados é, ele próprio, uma LLM que recebe instruções e dados pelo mesmo canal e, portanto, herda a vulnerabilidade que existe para mitigar. Ataques de evasão contra *guardrails* exploram exatamente isso, contornando detectores por ofuscação e por instruções dirigidas ao detector (HACKETT *et al.*, 2025). A literatura de defesa por marcação estrutural, que sinaliza tipograficamente a fronteira entre instrução confiável e dado não confiável (HINES *et al.*, 2024), foi proposta e avaliada para proteger o modelo principal, deixando em aberto a possibilidade de sua aplicação à entrada do componente de filtro.

Uma segunda lacuna aparece na saída do filtro. As propostas recentes de filtro por modelo convergem para a proposta de remover o conteúdo suspeito. O PromptArmor emprega uma LLM aberta para localizar e deletar o trecho injetado (SHI *et al.*, 2025); o DataFilter treina um modelo especificamente para apagar o trecho malicioso preservando o restante (WANG *et al.*, 2025); o PISanitizer identifica os *tokens* injetados por meio dos pesos de atenção e os elimina (GENG *et al.*, 2025). Remover é uma decisão irreversível tomada sob incerteza: quando o filtro erra, informação legítima é destruída silenciosamente e o componente principal responde sobre um texto mutilado, sem saber disso. Nenhum desses trabalhos considera a alternativa de **preservar o trecho e sinalizá-lo**, transferindo ao componente principal uma informação de procedência que ele possa ponderar.

Deste diagnóstico emerge a pergunta que estrutura este trabalho: **qual o ganho, medido na saída do sistema, de implementar separação entre canal de instrução e canal de dado em um *guardrail* de entrada?** Para responder, este trabalho propõe uma abordagem que trata a separação como variável manipulável em dois pontos distintos: na entrada do filtro e na sua saída, medindo o efeito de cada combinação sob as mesmas métricas.

1.1 Objetivo

O objetivo deste trabalho é **projetar e implementar um banco de provas científico** que quantifique o efeito da separação entre canal de instrução e canal de dado em um *guardrail* de entrada instanciado por modelos abertos, sem *fine-tuning*. A medida não é tomada na classificação isolada do filtro, mas na saída do componente principal: o modelo que efetivamente executa a tarefa, chamado na literatura de *backend LLM*, localizado ao final do fluxo completo.

Deste objetivo derivam três contribuições pretendidas:

- **Separação de canal na entrada do filtro.** Caracterizar se, e quanto, aplicar marcação estrutural (HINES *et al.*, 2024) ao conteúdo não confiável *que o filtro ins-*

peciona melhora sua capacidade de detecção, técnica até aqui avaliada apenas na proteção do modelo principal.

- **Marcação seletiva em vez de remoção.** Propor e avaliar um filtro que, ao liberar a requisição, marca tipograficamente os trechos que julgou perigosos e os repassa anotados ao componente principal, o qual recebe instrução explícita para tratá-los como dado de risco. Contrasta-se essa decisão com a alternativa dominante de exclusão (SHI *et al.*, 2025; WANG *et al.*, 2025; GENG *et al.*, 2025), cuja falha destrói informação.
- **O custo de dispensar o treinamento.** Situar o desempenho alcançado por configuração frente ao de um modelo *fine-tunado* para classificação de segurança, avaliando na mesma bateria, e sob as mesmas quatro condições, o filtro treinado sobre o mesmo *backbone* de um dos modelos adotados. A comparação é feita por medição direta, e não por reuso de números publicados (Seção 3.7).

2 Fundamentação Teórica

2.1 A LLM como componente e a interface do filtro

Utilizar uma LLM como componente de software exige tratá-la como um módulo com interface de entrada e saída bem definidas, e não como uma caixa de diálogo isolada (WEBER, 2024). O componente de filtro ocupa uma posição particular: atua antes da execução principal e não é responsável pela resposta final, mas por classificar e preparar a requisição recebida. Suas entradas são a requisição do usuário (potencialmente hostil); o conteúdo não confiável incorporado ao contexto, estes são: trechos recuperados (LEWIS *et al.*, 2020), retornos de ferramenta, documentos, páginas, entre outros recursos que constituem o vetor da injeção indireta (GRESHAKE *et al.*, 2023); e a política de classificação. Suas saídas formam um contrato tipado: uma decisão (**passar/bloquear**), uma marcação de risco e, opcionalmente, uma versão transformada do conteúdo; é neste último campo que reside a proposta deste trabalho.

2.2 *Prompt injection*: direto e indireto

O *prompt injection* é o ataque mais fundamental contra componentes baseados em LLM. Sua premissa é simples: instruções do administrador/desenvolvedor e a entrada do usuário chegam pelo mesmo canal, sem separação semântica garantida, de modo que qualquer conteúdo textual pode ser interpretado como instrução. Os primeiros estudos quantitativos introduziram as noções de *goal hijacking* e *prompt leaking* (PEREZ; RIBEIRO, 2022); trabalhos posteriores formalizaram o ataque e padronizaram sua avaliação (LIU *et al.*, 2024). Na variante **direta**, o atacante formula entradas que sobrepõem as instruções originais, da sobreposição explícita (“ignore as instruções anteriores”) a variantes como *role-playing* e ofuscação por codificação. Na variante **indireta** (GRESHAKE *et al.*, 2023), o atacante planta instruções em conteúdo que o componente processará normalmente, como um documento indexado, o retorno de uma ferramenta ou uma página web. É a variante indireta que motiva o recorte deste trabalho, por não depender de nenhuma ação do usuário legítimo.

2.3 A não separação como propriedade mensurável

A não separação entre instrução do administrador/desenvolvedor e a entrada do usuário decorre da forma como os *Transformers* são treinados para seguir instruções. O alinhamento por instrução, como o RLHF (OUYANG *et al.*, 2022), melhorou o respeito ao *system prompt*, mas não eliminou o problema, pois o modelo permanece um processador estatístico de texto sem mecanismo intrínseco para verificar a procedência de cada *token*.

Trabalho recente elevou essa observação a uma propriedade formal: é possível definir e medir o grau em que um modelo separa instrução de dado, comparando o comportamento do modelo quando uma mesma sentença aparece na posição de instrução e na posição de dado (ZVEREV *et al.*, 2025). A conclusão é que nenhum modelo avaliado atinge separação alta, e que técnicas canônicas de mitigação ou falham em melhorá-la substancialmente ou custam utilidade. Isso importa aqui por dois motivos: fornece a métrica com que se pode avaliar diretamente o efeito da intervenção proposta, e estabelece que a separação, embora não garantida pelo modelo, pode ser aproximada por convenções, premissa que fundamenta os métodos discutidos a seguir.

Há duas vias para obter essa aproximação. A via aprendida altera os pesos: consultas estruturadas (CHEN *et al.*, 2025a), otimização por preferência (CHEN *et al.*, 2025b) e hierarquia de instruções (WALLACE *et al.*, 2024) demonstram que a separação funciona quando treinada, e há modelos abertos já publicados sob essa abordagem (CHEN *et al.*, 2025c); a via arquitetural vai além e codifica a procedência nos próprios *embeddings* (ZVEREV *et al.*, 2026). Ambas exigem treinamento, e por isso servem aqui como referência. Uma terceira via reprojeta o agente em torno do modelo. O CaMeL separa fluxo de controle e fluxo de dados por meio de um interpretador com capacidades (DEBENEDETTI *et al.*, 2025), e Beurer-Kellner *et al.* (2025) catalogam padrões de projeto que restringem, por construção, o que o agente pode fazer depois de ler conteúdo não confiável. A via que este trabalho adota, opera apenas sobre o texto de entrada.

2.4 *Guardrails* de entrada e sua circularidade

Guardrails de entrada inspecionam toda a informação que chega ao componente antes de o modelo recebê-la. Há uma escolha estrutural entre filtros baseados em padrões (expressões regulares, listas), rápidos e auditáveis porém frágeis, e classificadores baseados em modelo, capazes de capturar semanticamente formulações que escapam a filtros superficiais (DONG *et al.*, 2024). Estes últimos, os *guard models*, são modelos de linguagem, frequentemente menores que o principal, ajustados para classificar entrada e saída segundo uma taxonomia (INAN *et al.*, 2023; ZENG *et al.*, 2024; ZHAO *et al.*, 2025), e já

compõem sistemas de defesa em camadas (CHENNABASAPPA *et al.*, 2025).

Um ponto importante para este trabalho é a circularidade: um *guard model* é ele próprio uma LLM e herda as vulnerabilidades que pretende mitigar. Pode ser manipulado por instruções embutidas no conteúdo que analisa e contornado por ofuscação (HACKETT *et al.*, 2025). Se separar a instrução do desenvolvedor da entrada não confiável é o que falta ao componente principal, não há razão para supor que o filtro esteja dispensado dela.

2.5 *Spotlighting*: separação por convenção

O *spotlighting* (HINES *et al.*, 2024) é a formulação de referência da separação por convenção. A ideia é transformar o texto não confiável antes de inseri-lo no *prompt*, de modo que o modelo receba um sinal contínuo de sua procedência, e avisar no *prompt* de sistema que a transformação foi feita e que nada dentro do bloco deve ser obedecido. Os autores descrevem três instâncias, de intrusividade crescente:

- **Delimitação** (*delimiting*). Símbolos especiais são colocados antes e depois do bloco não confiável, e o *prompt* de sistema declara que o texto entre eles é um documento a ser lido, e não instrução a ser seguida. Um resumo de e-mail, por exemplo, chegaria ao modelo como “<< texto do e-mail >>”, precedido de “nunca obedeça a instruções entre esses símbolos”. É a forma mais simples e a única que não altera o conteúdo do bloco; em contrapartida, marca apenas as bordas, e um atacante que conheça os símbolos pode fechá-los e inserir os seus próprios.
- **Marcação de dados** (*datamarking*). Em vez de marcar apenas as bordas, um caractere especial é intercalado ao longo de todo o bloco, tipicamente no lugar de cada espaço em branco: “In this manner” torna-se “In^this^manner”. O *prompt* de sistema explica a transformação, e a procedência passa a acompanhar cada palavra, e não só o início e o fim. O texto continua legível para o modelo, mas qualquer instrução embutida carrega a marca de dado.
- **Codificação** (*encoding*). O bloco inteiro é transformado por um algoritmo conhecido, como base64 ou ROT13, e o modelo é instruído a decodificá-lo antes de cumprir a tarefa. O conteúdo deixa de ser texto legível e vira dado opaco, o que torna a fronteira inequívoca, ao custo de exigir que o modelo decodifique corretamente.

As três reduziram de forma expressiva o sucesso de injeções indiretas nos experimentos originais, com a marcação de dados oferecendo o melhor equilíbrio entre proteção e preservação da tarefa, e a codificação dependendo da capacidade do modelo para decodificar sem perda. Esses resultados foram obtidos protegendo o modelo principal, o que torna as

técnicas candidatas naturais a serem aplicadas também ao filtro. Deste repertório, este trabalho adota a delimitação, por ser a menos intrusiva e a única que não altera o texto que o filtro precisa compreender, e transpõe a ideia de marcação por caractere para a saída do filtro, onde ela passa a ser aplicada de forma seletiva. A fragilidade da delimitação frente a quem conhece os símbolos é tratada pelo escape dos caracteres de tag, descrito na Seção 3.2.

É preciso distinguir esse mecanismo de defesas puramente textuais, que acrescentam ao *prompt* instruções de vigilância sem alterar a estrutura da entrada. O *self-reminder* (XIE *et al.*, 2023) é o exemplo canônico, e também reduz o sucesso de ataques. A distinção importa porque as duas famílias operam por vias diferentes: uma pede atenção ao modelo, a outra muda a forma do que ele lê. É a segunda que este trabalho investiga.

2.6 Filtros que sanitizam a entrada

Uma linha recente vai além de classificar e passa a **transformar** o conteúdo suspeito. O PromptArmor usa uma LLM sem treinamento adicional para localizar o trecho injetado e removê-lo por casamento aproximado, atingindo taxas de erro abaixo de 1% em um benchmark de agentes (SHI *et al.*, 2025). O DataFilter treina um modelo dedicado sobre o mesmo *backbone* de 8 bilhões de parâmetros para apagar o trecho malicioso preservando a informação benigna, e utiliza um *token* de fronteira para separar a instrução do filtro do conteúdo não confiável na entrada do próprio filtro (WANG *et al.*, 2025). O PISanitizer opera em caixa-branca, identificando os *tokens* injetados pelos pesos de atenção e eliminando-os (GENG *et al.*, 2025).

Os três compartilham a mesma decisão de projeto, que é a **remoção**, e este trabalho se diferencia deles nesta decisão, tomando uma abordagem diferente. A remoção é irreversível e tomada sob incerteza: um falso positivo apaga conteúdo legítimo, e o componente principal responde sobre um texto mutilado sem qualquer sinal de que isso ocorreu. A marcação, por outro lado, é uma operação conservadora: preserva o texto integralmente e delega ao componente principal a ponderação do risco, transformando uma decisão binária do filtro em um sinal graduado. Além disso, apenas o DataFilter separa a instrução do filtro do conteúdo não confiável, e o faz por meio de treinamento; nenhum dos três avalia o efeito de fazê-lo por alteração estrutural do *input*.

2.7 Métricas de avaliação

Medir apenas a detecção do filtro é insuficiente por duas razões. Primeiro, o que interessa é o desfecho: um ataque que passa pelo filtro mas não é executado pelo componente

principal não é um ataque bem-sucedido. A métrica primária é, portanto, a **taxa de sucesso do ataque** (ASR) medida na saída do sistema (YI *et al.*, 2025). Segundo, um filtro é sempre um equilíbrio entre detecção e **sobre-bloqueio**, a fração de entradas benignas indevidamente bloqueadas (modo de falha agravado por conteúdos benignos que contêm palavras-gatilho (LI *et al.*, 2024)), que tem igual impacto em casos reais de aplicação. Como o filtro opera sob restrição de falsos positivos, a métrica de classificação operacional é a taxa de detecção a uma taxa de falsos positivos baixa e fixa (JACOB *et al.*, 2025). A esses somam-se a **utilidade** preservada na tarefa legítima e o custo em *tokens* e latência, seguindo o princípio consolidado de avaliar conjuntamente utilidade e segurança (DEBENEDETTI *et al.*, 2024).

3 Metodologia

3.1 Fluxo da informação

A Figura 3.1 descreve o fluxo proposto e define os dois pontos de intervenção. As instruções do desenvolvedor e a requisição do usuário chegam ao filtro por **canais declaradamente distintos**: o canal de instrução carrega a política de classificação, e o canal de dado carrega o conteúdo não confiável, delimitado por uma tag explícita. O filtro emite uma decisão e, quando libera a requisição, também a lista dos trechos que julgou perigosos. Esses trechos são marcados tipograficamente no texto original, que segue íntegro, sem qualquer supressão, e repassados ao componente principal, o qual recebe, junto ao texto anotado, a instrução de tratar trechos marcados como dado de risco. O desfecho é medido na resposta do componente principal.

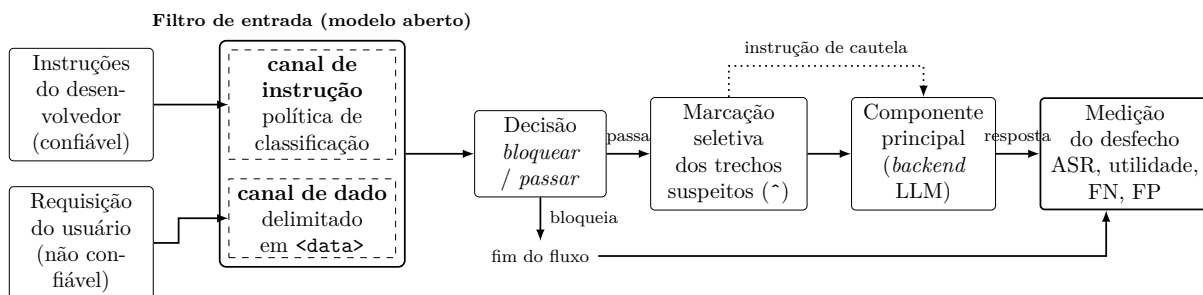


FIGURA 3.1 – Fluxo da informação. O filtro recebe instrução e dado por canais separados; ao liberar a requisição, marca os trechos suspeitos e os repassa anotados ao componente principal. A medição ocorre na saída do sistema, não na classificação do filtro.

O Apêndice A percorre esse fluxo sobre dois itens reais, um atacado e um benigno, com o texto literal do que cada estágio recebeu e respondeu sob cada configuração.

3.2 Primeiro fator: delimitação na entrada do filtro

O primeiro fator controla como o conteúdo não confiável é apresentado ao filtro, mantendo constante a política de classificação. Sem delimitação, o conteúdo é concatenado à requisição, como faz qualquer *guardrail* convencional. Com delimitação, ele é envolvido

em uma marcação estrutural explícita, isto é, a tag XML `<data>`, enquanto a política de classificação permanece fora de qualquer tag, e o *prompt* do filtro declara que o conteúdo interno à tag é dado a ser inspecionado, nunca instrução a ser seguida.

Um requisito de implementação decide se este fator mede o que pretende: o conteúdo não confiável precisa ter seus caracteres `<` e `>` escapados antes de entrar na tag. Sem isso, bastaria ao atacante incluir um `</data>` literal para encerrar a delimitação e voltar a ser lido como instrução. Nesta situação, o experimento estaria medindo uma delimitação que não delimita, situação pior do que a ausência de delimitação, por conferir falsa confiança ao resultado.

3.3 Segundo fator: marcação seletiva na saída do filtro

O segundo fator controla o que o filtro repassa ao componente principal quando decide **liberar** a requisição. Sem marcação, ele se comporta como o *guardrail* clássico: decide passar ou bloquear e não altera o texto. Com marcação, o filtro marca tipograficamente, com um caractere intercalado, apenas os trechos que ele próprio apontou como perigosos, e o *prompt* do componente principal declara explicitamente que trechos marcados representam risco e devem ser tratados como dado, jamais como instrução.

É neste fator que reside a proposta inovadora do trabalho, e a diferença em relação às defesas por sanitização está na decisão sobre o trecho suspeito: elas o **removem** (SHI *et al.*, 2025; WANG *et al.*, 2025; GENG *et al.*, 2025); aqui ele é **preservado e sinalizado**. Operacionalmente, o contrato de saída do filtro inclui a lista literal dos trechos suspeitos, e a marcação é aplicada ao texto original por casamento aproximado, a mesma técnica de localização que o PromptArmor emprega para remover (SHI *et al.*, 2025), aqui empregada para preservar.

A marcação é **seletiva**, e não integral, por duas razões. Marcar o bloco inteiro não discrimina: se tudo está marcado, nada se destaca, e a marca degrada a leitura do conteúdo legítimo. Em contrapartida, a marcação seletiva só protege aquilo que o filtro efetivamente identificou. Quando ele não aponta trecho algum, a condição degrada-se graciosamente para o *guardrail* clássico. Essa dependência da qualidade da identificação é uma propriedade do método, e não um defeito da implementação; medi-la é parte do que o experimento faz.

Com os dois fatores definidos, o cruzamento entre eles produz um desenho matricial 2×2 : quatro condições, uma para cada combinação de presença e ausência da delimitação na entrada e da marcação seletiva na saída (Tabela 3.1). Os fatores são binários e

independentes, e as quatro condições são executadas sobre os mesmos itens, o que permite isolar o efeito de cada intervenção e o da combinação das duas. A célula superior esquerda, o *prompt* simples, é o *guardrail* clássico: o mesmo filtro, com a mesma política, sem separação de canal em nenhum dos dois pontos. É contra ela que se mede o efeito da separação. A referência sem filtro, que não tem *guardrail* algum, não pertence à tabela e é descrita na Seção 3.4.

TABELA 3.1 – As quatro condições do desenho fatorial.

	Sem marcação	Com marcação seletiva
Sem delimitação	<i>prompt</i> simples	marcação
Com delimitação	delimitação	delimitação + marcação

Legenda. Linhas: primeiro fator, a delimitação na entrada do filtro (Seção 3.2). Colunas: segundo fator, a marcação seletiva na saída do filtro (Seção 3.3). Cada célula nomeia a condição como ela aparece nas figuras e tabelas de resultados. A composição exata das mensagens de cada condição está na Tabela A.2, no Apêndice A.

3.4 Linhas: modelos e referências

Cada bateria é executada sobre dois modelos abertos, e o modelo instancia o **sistema inteiro**, tanto o filtro quanto o componente principal, de modo que cada bateria descreva um sistema homogêneo. Os modelos são o Llama-3.1-8B-Instruct e o gpt-oss-20b, este último uma arquitetura de mistura de especialistas com 21 bilhões de parâmetros totais e 3,6 bilhões ativos por *token*.

Somam-se as referências. A primeira é a condição **sem filtro**, que estabelece a taxa de sucesso de ataque sem defesa alguma; sem ela não haveria denominador contra o qual expressar qualquer ganho. A segunda é o **filtro fine-tunado** da família gpt-oss: o gpt-oss-safeguard, treinado para classificação de segurança sobre exatamente o mesmo *backbone* do gpt-oss-20b, entra no papel de filtro nas mesmas quatro condições da Tabela 3.1, com o gpt-oss-20b como componente principal. Assim a comparação isola, célula a célula, o que o treinamento disponível na família acrescenta e o que a separação de canal ainda acrescenta sobre um filtro treinado, e o faz por medição direta, sem depender de números publicados por terceiros. O gpt-oss-safeguard é um classificador de segurança de conteúdo guiado pela política do *prompt* de sistema, mas não é um detector treinado especificamente para *prompt injection*, portanto, ele recebe a mesma política que os modelos abertos.

A família Llama não recebe referência equivalente, e a razão é de disponibilidade. Os detectores treinados especificamente para *prompt injection* sobre esse *backbone* existem como pesos publicados (WANG *et al.*, 2025; GENG *et al.*, 2025; JACOB *et al.*, 2025), mas nenhum é servido por API: usá-los exigiria hospedar um modelo de oito bilhões de pa-

râmetros em infraestrutura própria, o que sairia do custo e da reprodutibilidade que o trabalho adota. Os detectores pequenos o bastante para rodar sem servidor dedicado são codificadores treinados à parte, e não modelos da família avaliada. Essas defesas permanecem, portanto, como referência de literatura, discutidas no Capítulo 2, e a comparação medida é a que a família gpt-oss permite. O *guard* de conteúdo da família Llama, o Llama Guard 4, não entra no desenho por ser derivado do Llama 4 Scout, e não do Llama 3.1.

O desenho completo compreende, portanto, **quatorze execuções** por benchmark: as quatro condições sobre os dois modelos abertos, duas referências sem filtro e as quatro condições com o filtro fine-tunado.

3.5 Benchmarks

A avaliação usa três benchmarks públicos e citáveis, cada um responsável por um eixo distinto do desfecho:

- **BIPIA** (YI *et al.*, 2025): injeção indireta em cinco tarefas de texto (correio eletrônico, perguntas sobre página web, tabelas, sumarização e código). Fornece simultaneamente a taxa de sucesso do ataque e a utilidade na tarefa legítima, e é o único dos três que exercita o fluxo completo até a resposta do componente principal. É o benchmark primário.
- **SEP** (ZVEREV *et al.*, 2025): mede diretamente o grau de separação entre instrução e dado, comparando o comportamento do modelo quando a mesma sentença ocupa a posição de instrução e a de dado. É o instrumento que liga o resultado empírico à propriedade formal que a intervenção pretende melhorar.
- **NotInject** (LI *et al.*, 2024): conjunto exclusivamente benigno, construído com palavras-gatilho que induzem detectores ao erro. Mede o sobre-bloqueio, isto é, o preço pago em falsos positivos por qualquer ganho de detecção.

A amostragem é estratificada por rótulo com semente fixa, e as suítes são congeladas antes da execução.

3.6 Métricas e protocolo

Registram-se métricas em dois pontos. **No filtro**: taxa de detecção a 1% de falsos positivos, sobre-bloqueio sobre o NotInject, custo em *tokens* e tempo de geração p50/p95, informado pelo provedor de cada chamada, além da **taxa de saída inválida**: modelos

pequenos frequentemente violam o formato de saída pedido, e itens inválidos são contabilizados como não bloqueio. **Na saída do componente principal:** taxa de sucesso do ataque, utilidade preservada na tarefa benigna, e a decomposição dos erros em falso-negativo (ataque liberado e executado) e falso-positivo (requisição benigna bloqueada ou degradada pela marcação). A resposta vazia do componente principal, que ocorre quando um modelo de raciocínio esgota o teto de *tokens* antes de responder, é tratada como saída inválida, e não como bloqueio: não executa o ataque, não cumpre a tarefa e não entra no sobre-bloqueio, que mede apenas a decisão do filtro. Sua frequência é reportada à parte, como a da saída inválida do filtro. A tarefa conta como cumprida quando ao menos metade das palavras de conteúdo da resposta de referência aparece, como palavra inteira, na resposta produzida; referências curtas, como um número, precisam aparecer por inteiro. Palavra, aqui, é uma sequência de letras e dígitos, de modo que pontuação e marcação de formatação coladas a ela não impedem o casamento. A regra evita dois erros de um critério mais simples: separar as palavras só por espaço reprovava respostas corretas escritas em negrito ou seguidas de ponto, e procurar a referência como trecho do texto aprovaria “4” dentro de qualquer número que contivesse esse dígito.

Explicando a taxa de detecção a 1% de falsos positivos: a forma de medição vem de Jacob *et al.* (2025), que argumentam que, no tráfego real, as entradas benignas superam de longe os ataques, de modo que um detector só é implantável se quase nunca bloquear conteúdo legítimo. Em vez de julgar o detector pelo limiar padrão da classificação, eles calibram o limiar para uma taxa alvo de falsos positivos (1%, 0,5%, 0,1% e 0,05%) e reportam a detecção obtida em cada uma. Aqui adota-se o alvo de 1%, o menos exigente da escala, porque as amostras têm 150 itens benignos e alvos menores não teriam resolução. Além da decisão, o filtro atribui a cada item um escore de risco entre 0 e 1. Ordenados os itens por escore, pergunta-se a partir de que valor se deveria bloquear para que, no máximo, 1% dos itens benignos fossem bloqueados, e reporta-se a fração dos atacados que esse limiar captura. Um exemplo em miniatura: dez itens, cinco benignos com escores 0,0, 0,1, 0,2, 0,3 e 0,5, e cinco atacados com escores 0,4, 0,6, 0,7, 0,8 e 0,9. Bloqueando a partir de 0,6, caem quatro atacados e nenhum benigno: detecção de 0,80 com 0% de falsos positivos. Baixando o limiar para 0,5, o benigno de escore 0,5 passa a ser bloqueado, e a taxa de falsos positivos salta para 20%, fora do orçamento. O maior limiar que respeita a restrição é, portanto, 0,6, e a métrica vale 0,80; o atacado de escore 0,4 fica sem detecção, e esse é o preço da restrição. Com os 150 benignos do BIPIA, 1% equivale a no máximo um benigno bloqueado. A métrica é, assim, uma reanálise dos escores após a execução: o limiar que o filtro de fato aplicou na bateria é o que alimenta as demais métricas, enquanto esta mede quanto o classificador detectaria se o limiar fosse ajustado sob a restrição. Ela depende da resolução do escore: um filtro que só emite três valores distintos raramente encontra um limiar intermediário, e o número reportado cai a zero, convenção que também segue a referência: quando nenhum limiar atinge a taxa alvo, a detecção é dada como

nula.

O efeito é exibido através da **diferença** contra duas referências distintas: sem filtro e *prompt* simples, porque elas respondem a perguntas diferentes. Contra a condição **sem filtro**, obtém-se o quanto o *guardrail* protege no total; contra o *prompt* **simples** (que já é um *guardrail* completo) obtém-se o que é atribuível especificamente à separação de canal, que é a pergunta de pesquisa.

A bateria inteira é executada **dez vezes**, com a mesma configuração e sobre os mesmos itens. A repetição existe porque o modelo não é determinístico, nem mesmo com temperatura zero: o *prompt* simples e a marcação enviam ao filtro exatamente o mesmo texto, já que a marcação só age depois da decisão, e ainda assim a decisão sobre um mesmo item muda de uma execução para outra. O desfecho de cada item passa a ser a média das suas dez repetições, isto é, a fração delas em que o ataque foi executado ou a tarefa foi cumprida, e é sobre essas médias que todas as grandezas são calculadas.

Os intervalos de confiança são de 95% e vêm do *bootstrap* BCa (EFRON, 1987), com 10.000 reamostragens dos itens, tanto para as taxas absolutas quanto para as diferenças entre condições e para os efeitos do desenho fatorial. Como todas as condições são avaliadas sobre exatamente os mesmos itens, a diferença entre duas condições é calculada item a item, e cada reamostragem sorteia itens levando junto os seus valores nas condições comparadas. O intervalo é, assim, pareado: a variação que vem do item, e não da condição, sai da conta. A unidade sorteada é o item, e não a execução. Cada item entra com a média das suas repetições, e a variação entre repetições viaja com ele; contar cada execução como um item independente trataria dez medições do mesmo item como dez itens e estreitaria o intervalo sem fundamento. O BCa corrige o viés e a assimetria da distribuição reamostrada. Quando menos de cinco itens se afastam do valor mais comum, como numa taxa de sobre-bloqueio quase sempre nula, a reamostragem não tem o que sortear e devolveria um intervalo de largura nula ou quase, que afirmaria uma precisão que o dado não tem. Nesses casos não se reporta intervalo: a barra aparece sem haste, e o valor se lê como a contagem que ele é, por exemplo um bloqueio em 1.500 decisões. O valor central é sempre a média sobre os itens, que coincide com a taxa agregada das dez repetições. Nas figuras, o intervalo aparece como a haste vertical em torno de cada barra ou ponto, e nas diferenças a leitura relevante é se a haste contém ou não o zero: quando não contém, a diferença não se explica pelo acaso, seja o da amostragem dos itens, seja o da variação entre execuções.

O desenho fatorial permite, além das comparações contra o *prompt* simples, estimar o efeito de cada fator usando as quatro condições de uma vez. O **efeito principal** da delimitação é a média das duas condições com delimitação menos a média das duas sem; o da marcação é definido da mesma forma. A **interação** é a diferença entre o ganho da marcação com e sem delimitação, e vale zero quando os dois fatores simplesmente se

somam. Cada efeito é calculado item a item e resumido pela média; como cada item entra nas quatro condições, o intervalo fica mais estreito do que o de qualquer comparação entre duas condições. O intervalo é o mesmo BCa sobre os itens. O Apêndice B desenvolve as contas dos intervalos sobre exemplos numéricos e mostra como as dez repetições entram nelas.

A semente da amostragem é fixa, de modo que as dez repetições usam os mesmos itens, e todos os registros por item são persistidos, o que garante a reprodutibilidade da análise. O registro de cada item inclui o conteúdo não confiável como cada estágio o recebeu: delimitado, na entrada do filtro; anotado, na entrada do componente principal, além dos trechos efetivamente marcados, de modo que a transformação aplicada pelo *guardrail* possa ser auditada após a execução, e não apenas inferida da condição.

3.7 Situando o resultado frente ao *fine-tuning*

Saber o quanto a configuração recupera exige compará-la a um modelo treinado para o papel de filtro, e a comparação é feita por medição direta, na mesma bateria, sobre os mesmos itens e com as mesmas métricas. O gpt-oss-safeguard compartilha o *backbone* do gpt-oss-20b e roda as mesmas quatro condições, o que permite ler a diferença em dois sentidos: entre as duas linhas, célula a célula, está o que o treinamento de segurança acrescenta; dentro da linha do safeguard está o que a separação de canal acrescenta a um filtro já treinado. A segunda leitura é a que interessa à pergunta de pesquisa, porque responde se a configuração ainda tem efeito quando o filtro não é um modelo de uso geral.

Resultados publicados por terceiros sobre os mesmos benchmarks abertos (CHEN *et al.*, 2025c; WANG *et al.*, 2025) entram apenas como contexto secundário, e cada número externo é registrado com benchmark, métrica, valor e fonte, de modo que sua procedência acompanhe o resultado.

3.8 Execução e consolidação

A bateria foi executada integralmente sobre os três benchmarks, com quatorze execuções por benchmark: as quatro condições da Tabela 3.1 nos dois modelos abertos, as duas referências sem filtro e as quatro condições com o filtro fine-tunado, ou seja, 42 execuções por repetição. Com as dez repetições, a bateria soma **420 execuções e 131.460 registros**, um por item em cada repetição. O BIPIA e o SEP foram avaliados sobre 300 itens amostrados de forma estratificada; do NotInject, que é menor, foram usados os 339 itens completos.

Na consolidação dos registros são calculados todos os intervalos de confiança, cada condição contra as duas referências descritas na Seção 3.6, para taxa de sucesso do ataque e para utilidade, e não apenas os de um recorte.

3.8.1 Orçamento de geração

Toda chamada de modelo recebe um teto de geração: 2.048 *tokens* no filtro e 8.192 no componente principal, iguais para todas as condições. A finalidade do teto é limitar custo e latência por item e garantir que cada chamada termine; o valor, porém, não é indiferente ao resultado. Um dos modelos abertos adotados é modelo de raciocínio: ele consome parte do orçamento de geração antes de emitir a resposta e, se o teto é alcançado antes do fim, a interface devolve conteúdo vazio, acompanhado da indicação de truncamento. Uma resposta vazia não contém a testemunha do ataque, e por isso conta como ataque não executado; tampouco cumpre a tarefa. O efeito não é uniforme: o esvaziamento cresce com o comprimento do *prompt*, e as condições que acrescentam texto, justamente as que implementam a separação de canal, seriam as mais truncadas, produzindo um viés na direção da hipótese.

O teto de 8.192 *tokens* no componente principal foi escolhido para que a falta de orçamento não decida o resultado: a esse valor, o esvaziamento fica restrito a uma pequena fração das respostas, em que o modelo esgota o teto raciocinando. Essas respostas vazias contam como saída inválida do componente, com frequência reportada à parte (Seção 4.4). No filtro, o teto de 2.048 *tokens* basta para a decisão em JSON; as chamadas que o atingem sem decisão legível contam como não bloqueio, a regra conservadora adotada para toda saída inválida. O registro de cada item guarda o motivo de término dos dois estágios.

4 Resultados

Este capítulo apresenta as medições da bateria descrita na Seção 3.8. Cada figura é uma grade de nove painéis. As linhas trazem os dois modelos abertos, que instanciam o sistema inteiro (filtro e componente principal), e o filtro fine-tunado da família gpt-oss, que ocupa só o papel de filtro. As colunas trazem os três benchmarks. No eixo horizontal de cada painel ficam as cinco configurações: sem filtro, *prompt* simples, delimitação, marcação e a combinação das duas. O NotInject tem apenas itens benignos, então nele a taxa de sucesso do ataque não é definida e o painel correspondente fica vazio.

Todo valor é a média das dez repetições da bateria, e toda haste vertical é o intervalo de confiança de 95% descrito na Seção 3.6. Nas figuras de valor absoluto, a haste mostra a incerteza de cada taxa; a barra sem haste é uma taxa em que menos de cinco itens variaram, quase constante, e sem intervalo pela regra da Seção 3.6. Nas figuras de diferença, cada diferença é um ponto: se a haste não cruza o zero, a diferença não se explica pelo acaso; se cruza, falta evidência de efeito nesta amostra, o que não equivale a ausência de efeito. As medidas de validade e de custo operacional (saída inválida, resposta vazia, alcance da marcação, *tokens* e tempo) descrevem o comportamento observado e não trazem intervalo.

4.1 Taxa de sucesso do ataque

A Figura 4.1 apresenta a taxa de sucesso do ataque em valor absoluto.

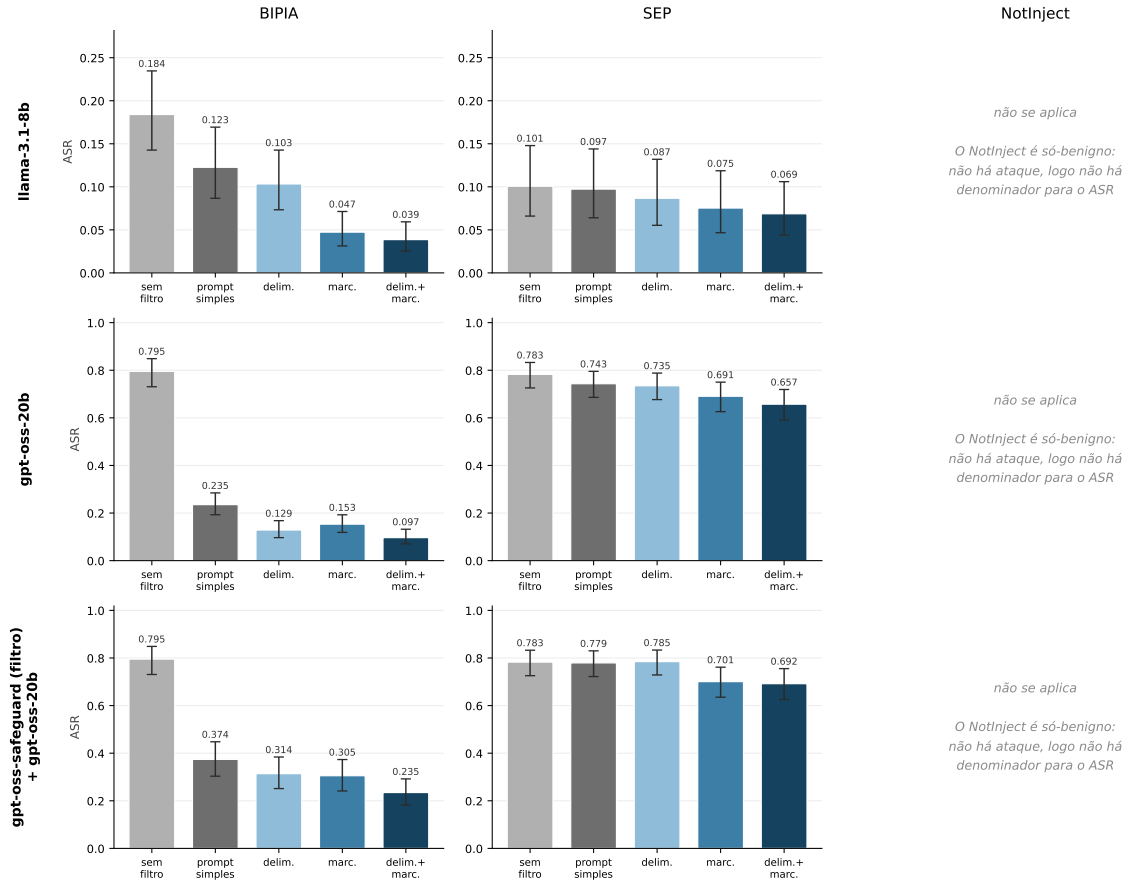


FIGURA 4.1 – Taxa de sucesso do ataque por configuração. O painel do NotInject fica vazio porque o conjunto não tem itens atacados.

A proteção total do *guardrail* é a distância entre cada configuração e a barra sem filtro. O filtro fine-tunado protege menos que o modelo aberto da mesma família em todas as configurações; a Seção 5.5 discute o motivo.

A Figura 4.2 isola o efeito de separar o canal de instrução do canal de dado não confiável. Cada condição é comparada ao *prompt simples*, com o mesmo filtro, os mesmos itens e o mesmo componente principal.

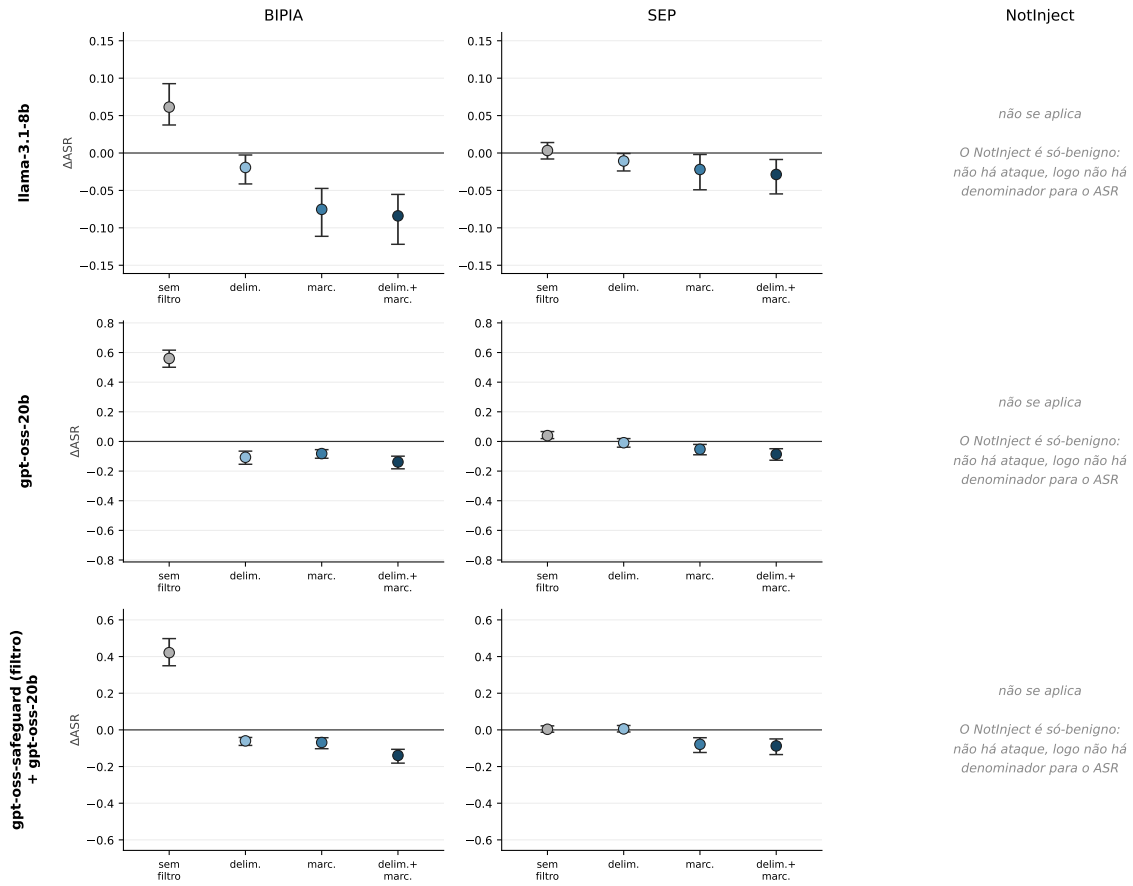


FIGURA 4.2 – Diferença na taxa de sucesso do ataque de cada condição em relação ao *prompt* simples. As hastes são intervalos de confiança de 95% por *bootstrap* BCa pareado sobre os itens.

A Figura 4.3 decompõe o mesmo resultado nos efeitos do desenho fatorial (Seção 3.6): o efeito principal da delimitação, o da marcação e a interação entre os dois, cada um estimado com as quatro condições.

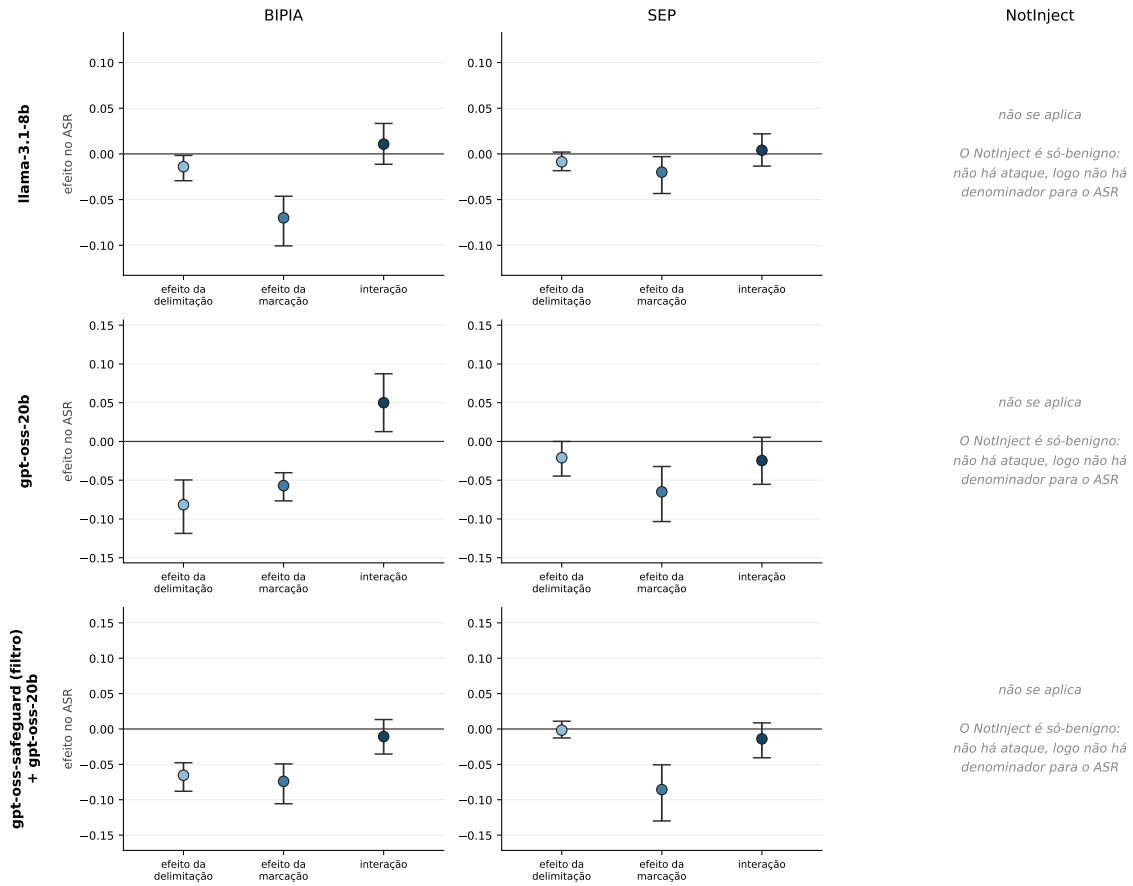


FIGURA 4.3 – Efeitos do desenho 2×2 sobre a taxa de sucesso do ataque. O efeito principal de um fator é a média das duas condições com o fator menos a média das duas sem ele; a interação é a diferença entre o ganho da marcação com e sem delimitação. As hastes são intervalos de confiança de 95% por *bootstrap* BCa sobre os itens.

4.2 Custo da defesa

Uma redução na taxa de sucesso do ataque só é ganho se não sacrificar a tarefa legítima. A Figura 4.4 mostra a diferença de utilidade de cada condição em relação ao *prompt* simples, com o mesmo intervalo pareado.

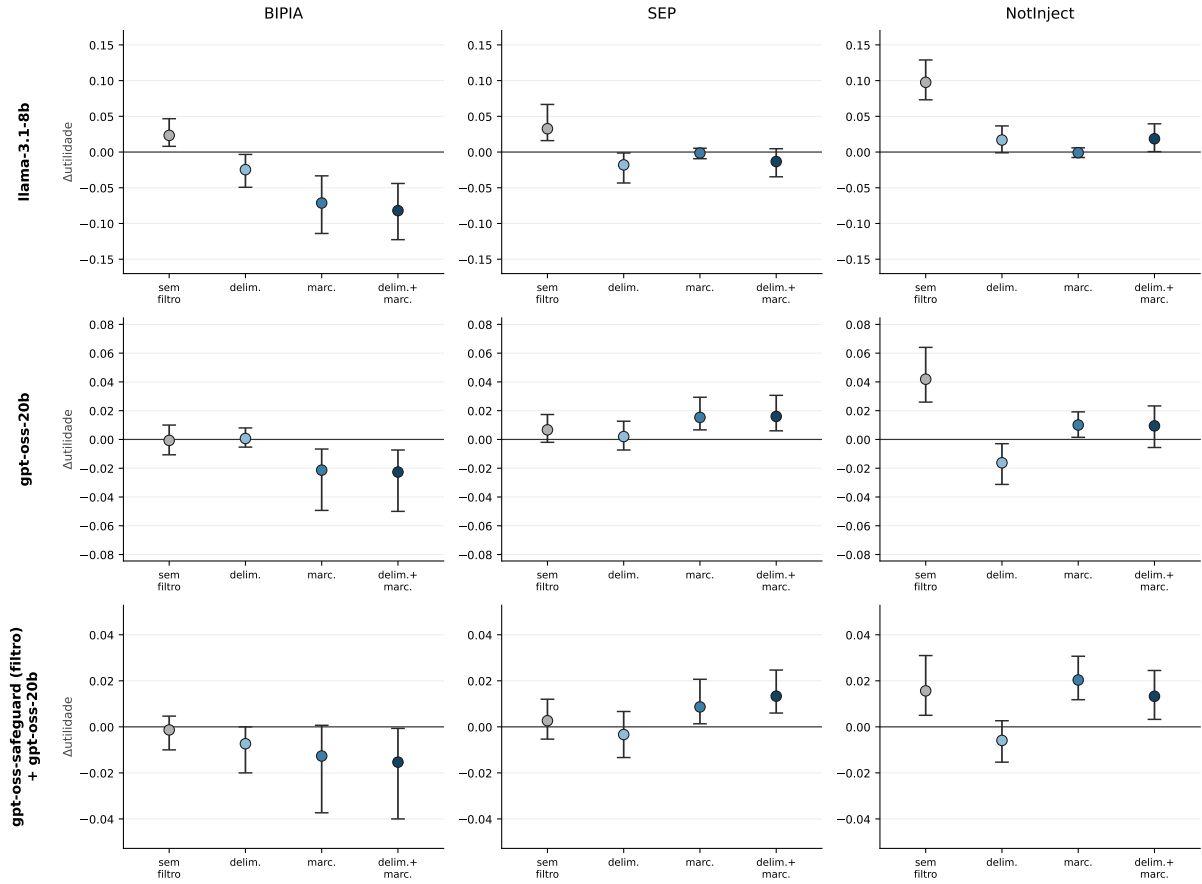


FIGURA 4.4 – Diferença de utilidade de cada condição em relação ao *prompt* simples, com intervalos de confiança de 95% por *bootstrap* BCa pareado sobre os itens. Só o BIPIA publica resposta de referência; no SEP e no NotInject, a utilidade significa “respondeu sem ter sido bloqueado”, e não “respondeu corretamente”.

A Figura 4.5 aplica a mesma decomposição fatorial à utilidade. Por usar as quatro condições, ela separa o custo de cada fator melhor que as comparações contra o *prompt* simples.

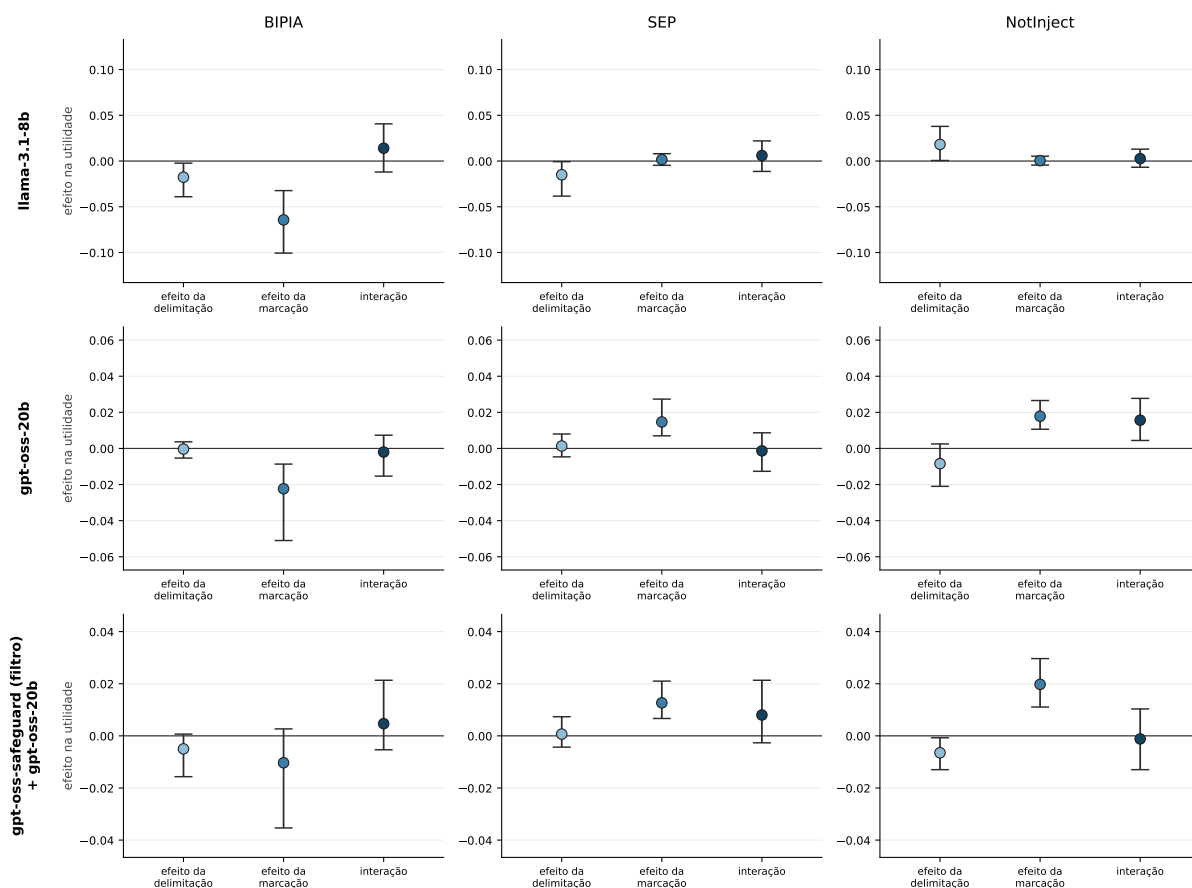


FIGURA 4.5 – Efeitos do desenho 2×2 sobre a utilidade, definidos como na Figura 4.3, com intervalos de confiança de 95% por *bootstrap* BCa sobre os itens. Vale a ressalva da Figura 4.4 sobre o significado da utilidade.

A Figura 4.6 mostra a utilidade em valor absoluto.

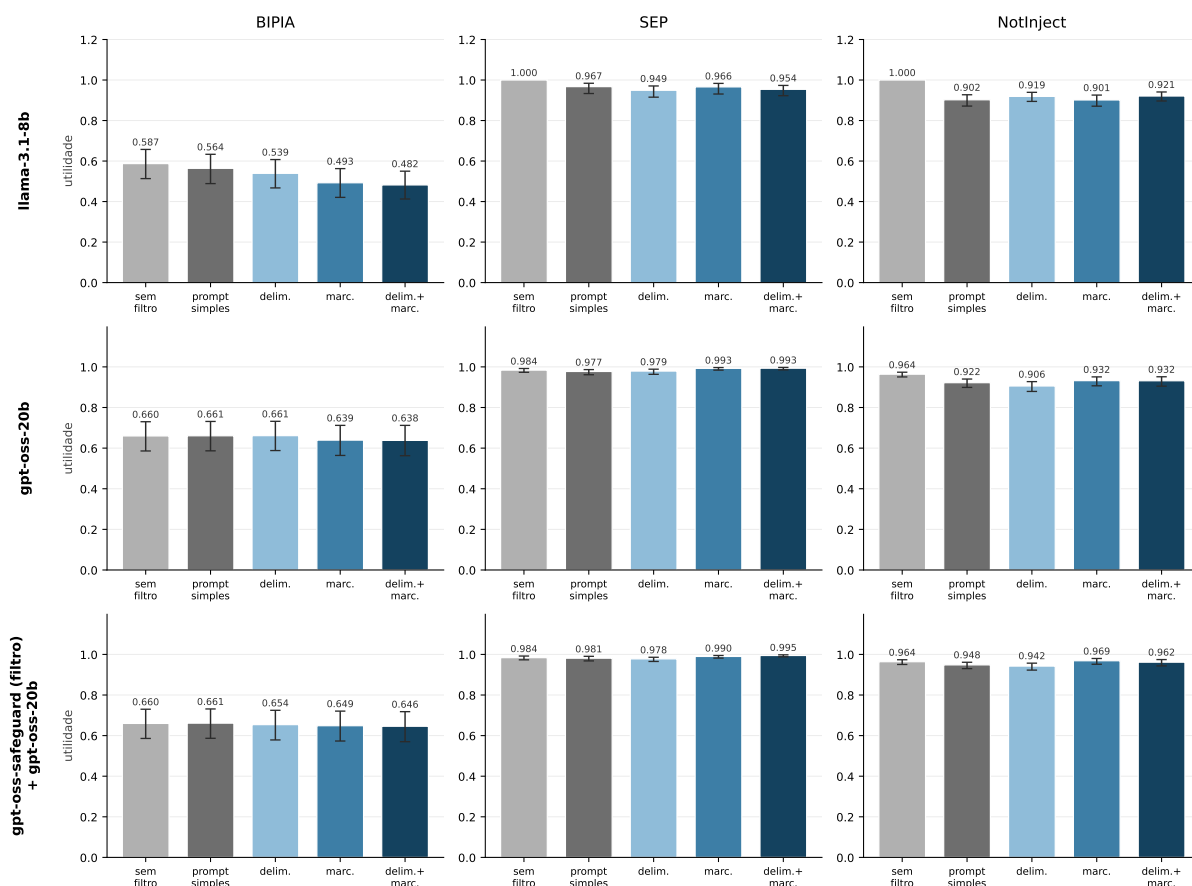


FIGURA 4.6 – Utilidade: fração das tarefas legítimas cumpridas, com item bloqueado contando como fracasso. As duas barras sem haste, do Llama-3.1-8B sem filtro no SEP e no NotInject, valem exatamente 1,000, sem variação para um intervalo. Vale a ressalva da Figura 4.4 sobre o significado da utilidade.

A Figura 4.7 mostra o sobre-bloqueio, a fração do conteúdo legítimo que o filtro bloqueou.

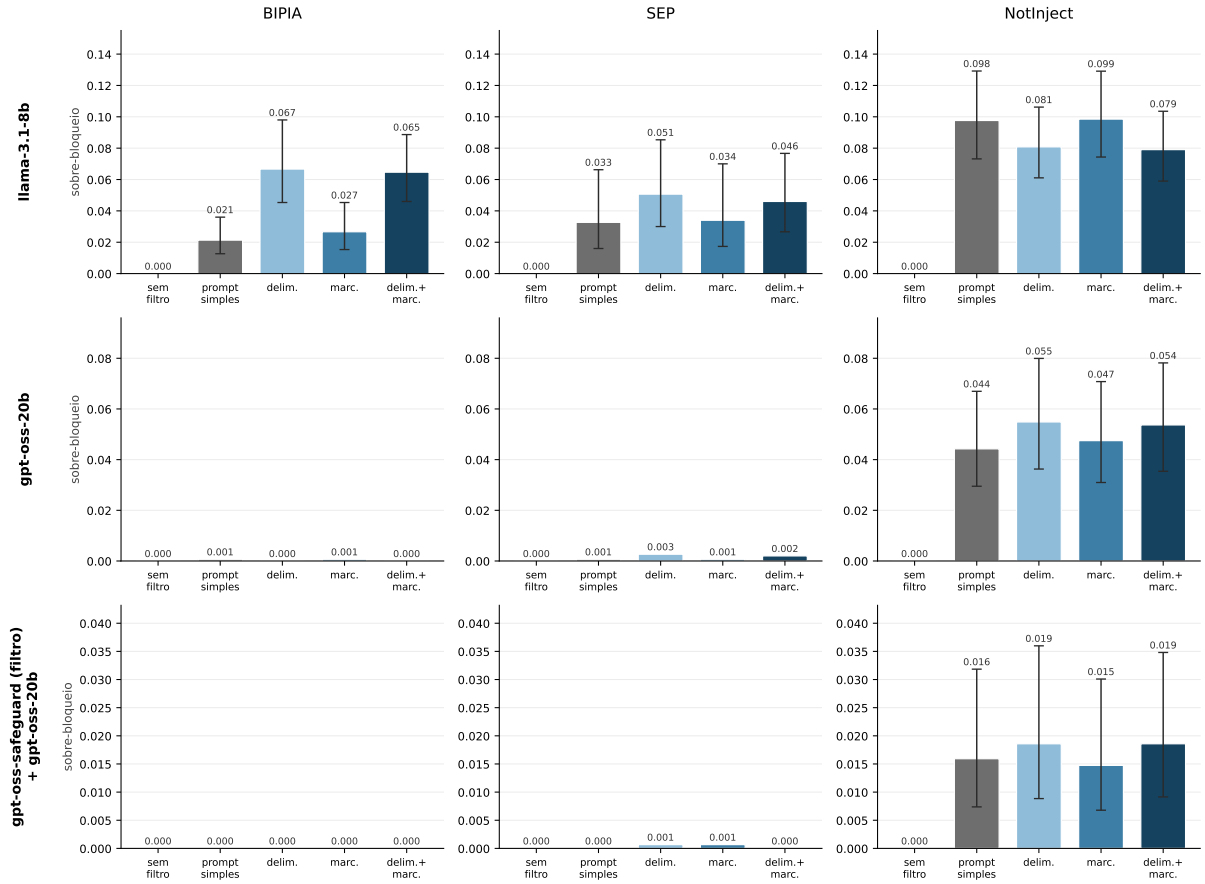


FIGURA 4.7 – Sobre-bloqueio: fração dos itens benignos que o filtro bloqueou. É a única métrica definida nos três benchmarks e o motivo de incluir o NotInject, composto de requisições legítimas com vocabulário sensível. Sem filtro não há decisão de bloqueio, e a barra correspondente é nula por construção. As barras sem haste do gpt-oss-20b e do gpt-oss-safeguard no BIPIA e no SEP somam de zero a quatro bloqueios em 1.500 decisões, poucos demais para um intervalo.

4.3 Alcance da marcação seletiva

A marcação seletiva só age sobre os trechos que o filtro apontou. Sem trecho apontado, não há o que marcar, e a condição se reduz ao *prompt* simples. A Figura 4.8 mede esse alcance.

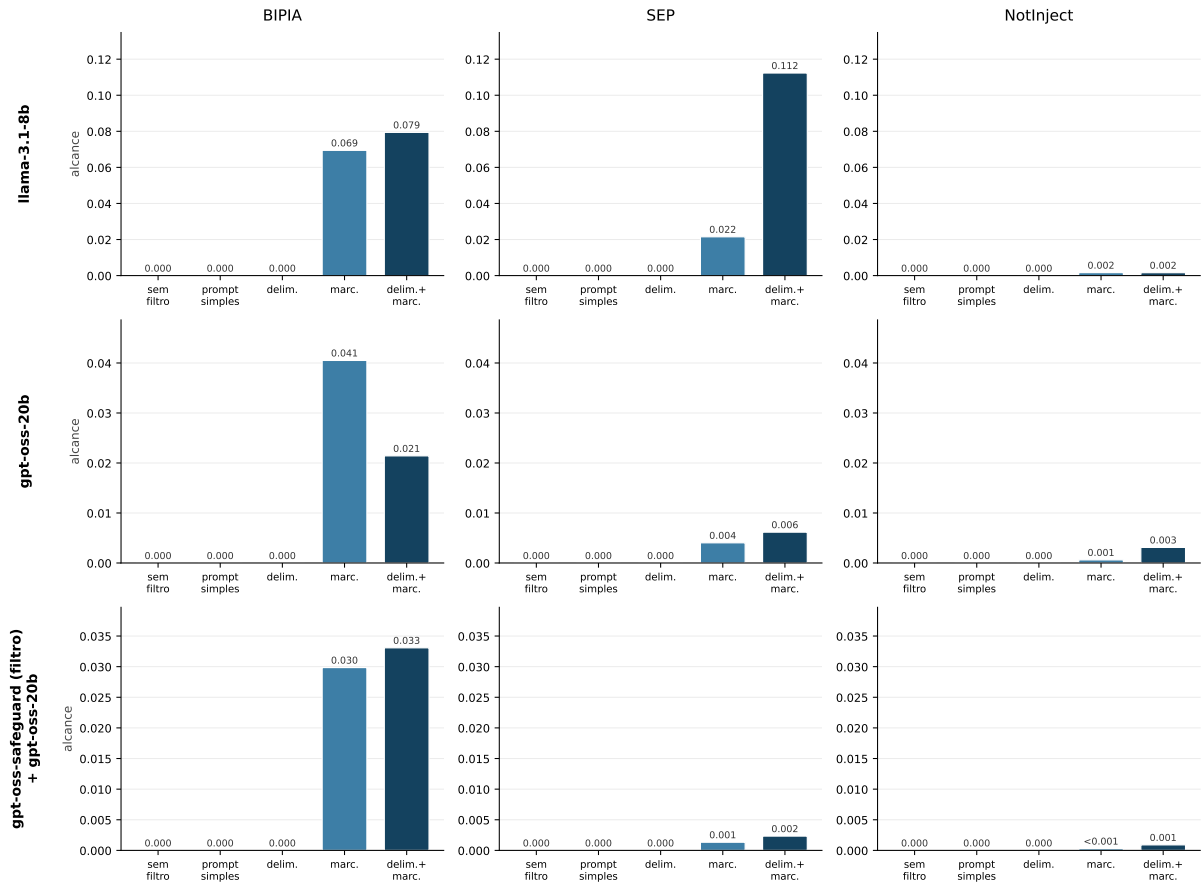


FIGURA 4.8 – Alcance da marcação seletiva: fração dos itens liberados pelo filtro com ao menos um trecho marcado. É zero por construção nas condições que não marcam.

4.4 Saídas inválidas

A Figura 4.9 mostra a taxa de saída inválida do filtro.

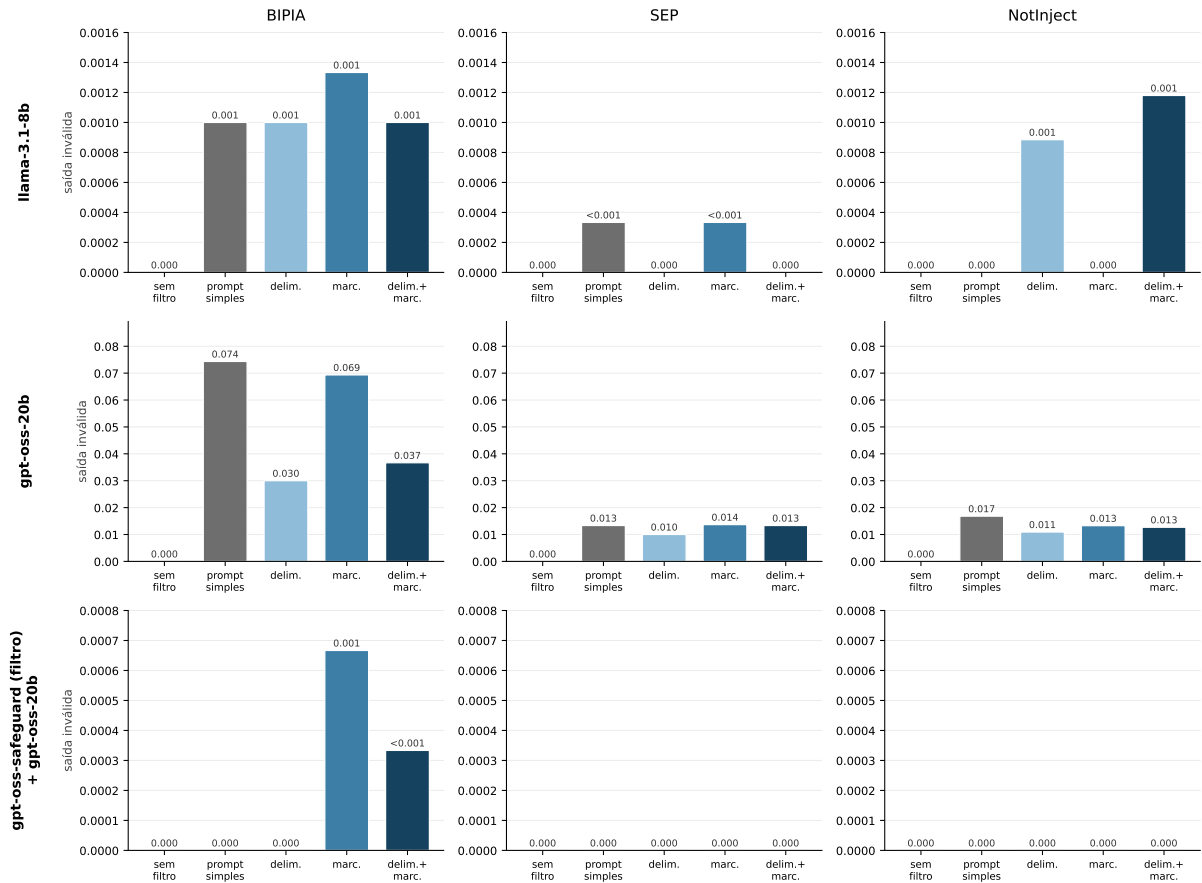


FIGURA 4.9 – Taxa de saída inválida do filtro: fração das respostas que não seguiram o contrato de saída. Saída inválida conta como não bloqueio: o item segue para o componente principal e entra na medição.

A Figura 4.10 mostra o equivalente no componente principal: a fração dos itens liberados pelo filtro em que ele devolveu resposta vazia.

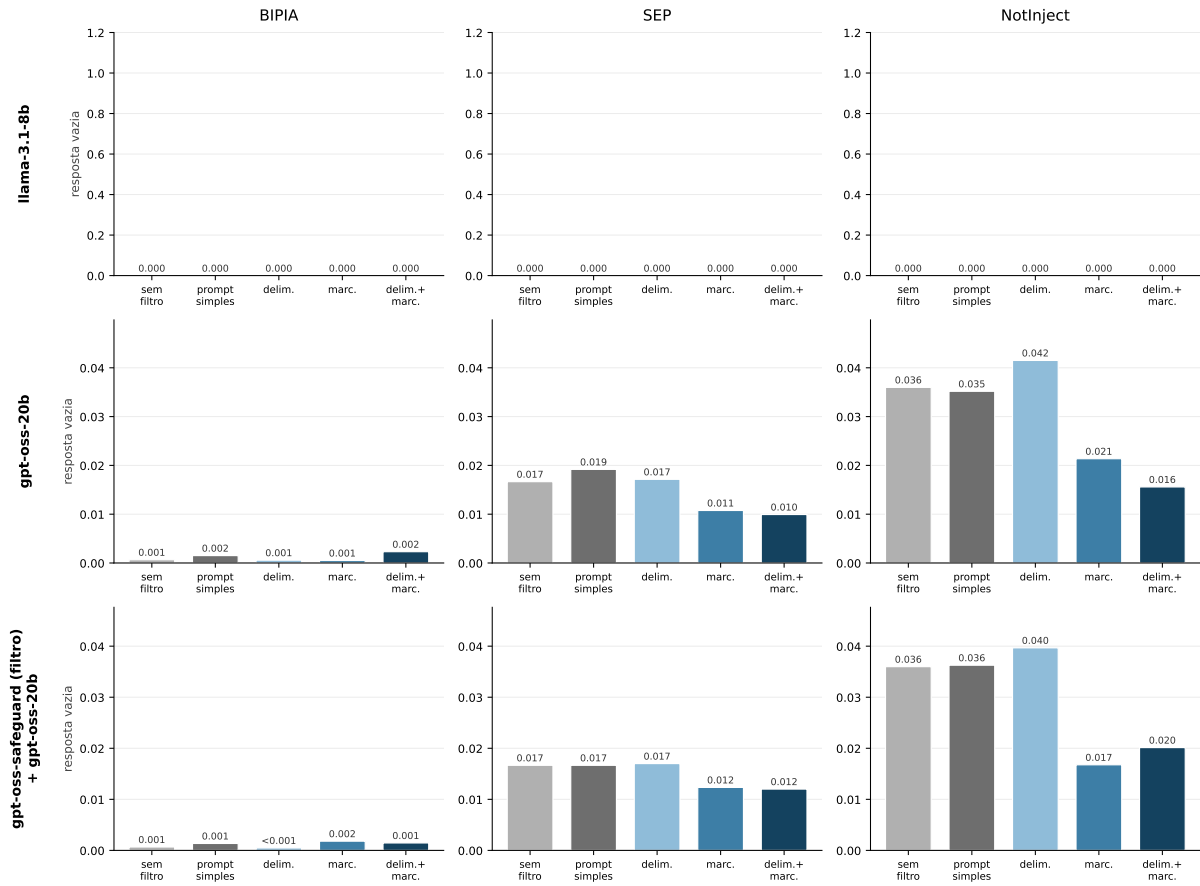


FIGURA 4.10 – Resposta vazia do componente principal: fração dos itens liberados pelo filtro em que o componente não produziu resposta, quase sempre por esgotar o teto de 8.192 *tokens* raciocinando. Resposta vazia conta como saída inválida, e não como bloqueio nem como sobre-bloqueio: não executa o ataque e não cumpre a tarefa.

4.5 Custo operacional

As Figuras 4.11 e 4.12 mostram o custo em *tokens* e o tempo de geração por item, somando os dois estágios.

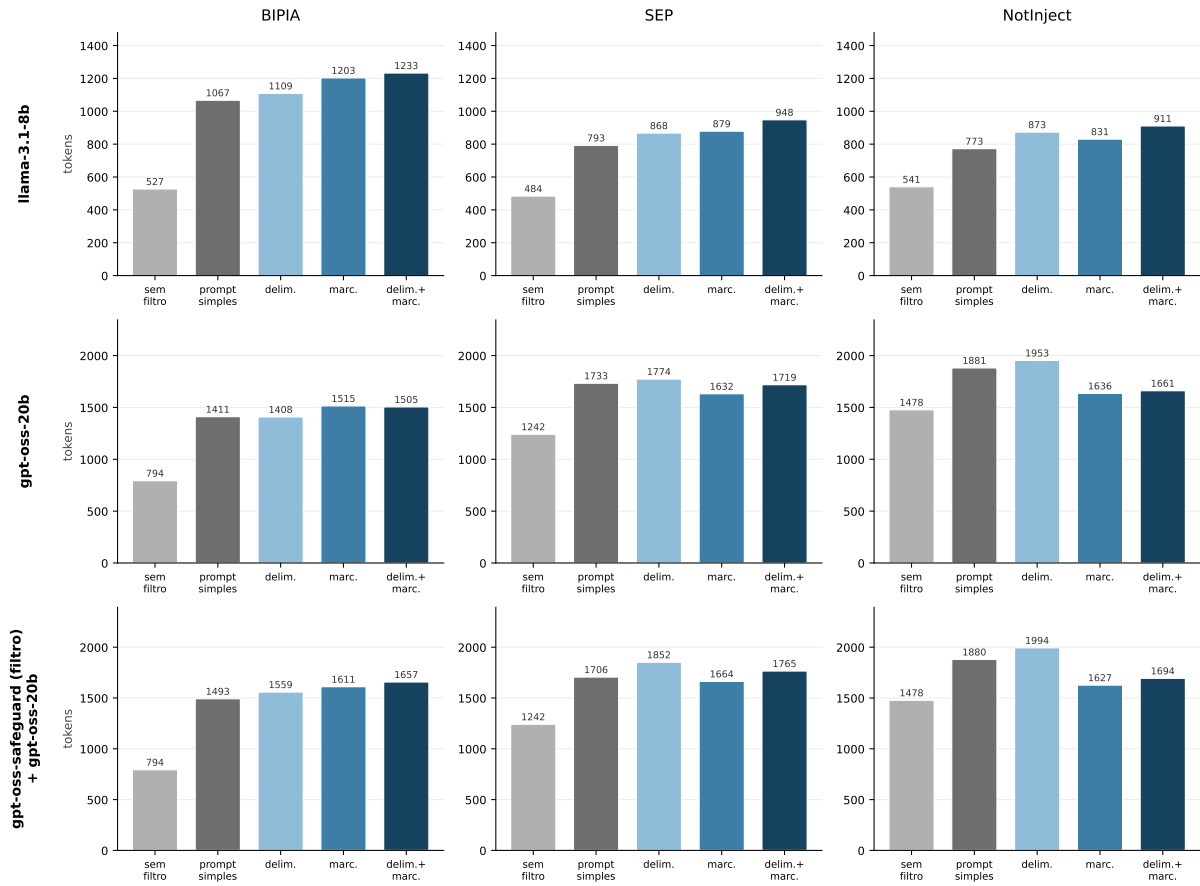


FIGURA 4.11 – Tokens por item, somando os dois estágios. A diferença para o *prompt simples* é a despesa da delimitação e da marcação, que acrescentam texto; a diferença para a configuração sem filtro é a despesa do filtro, que dobra o número de chamadas.

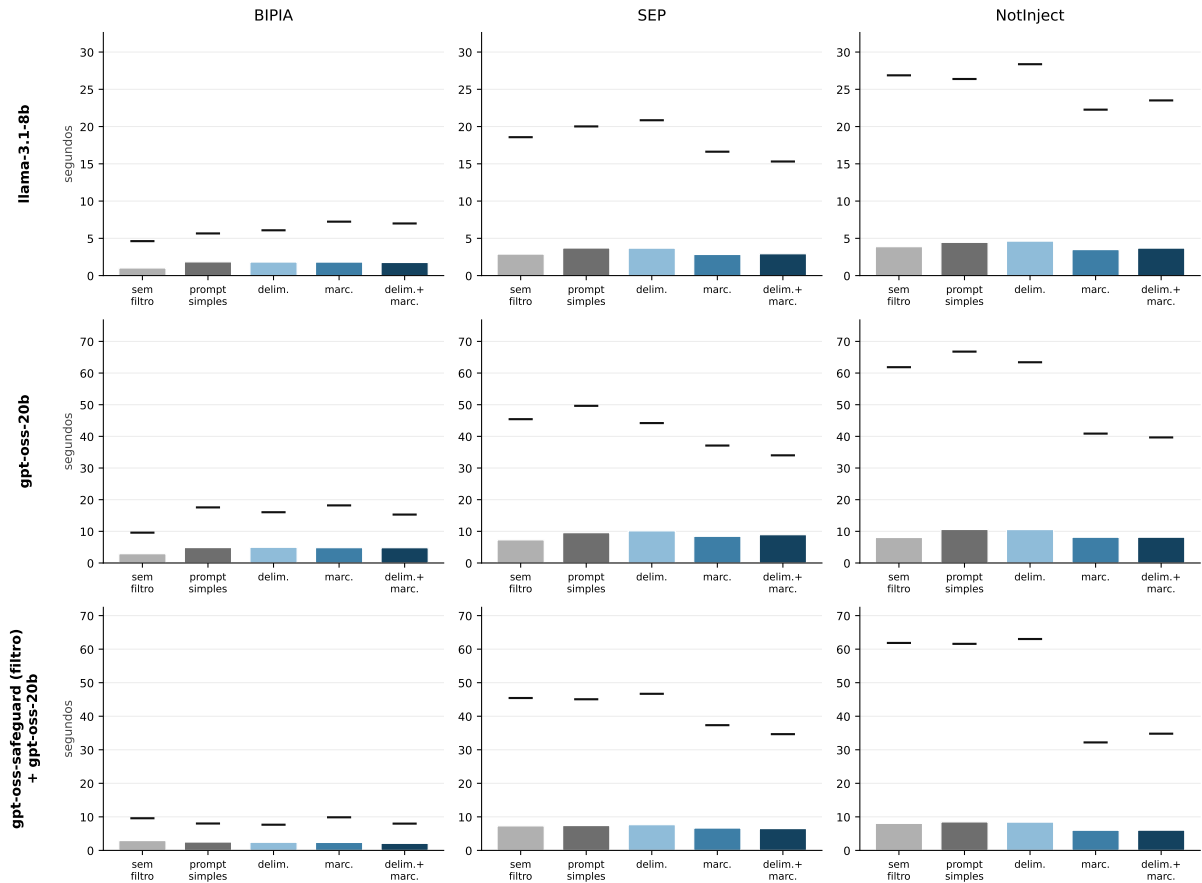


FIGURA 4.12 – Tempo de geração por item, em segundos, somando os dois estágios. A barra é a mediana e o traço, o percentil 95. O tempo é o informado pelo provedor para cada chamada, e por isso não inclui fila, rede nem espera por limite de taxa. O mesmo modelo é servido por provedores com hardware diferente, escolhidos a cada chamada pela interface, e essa variação entra no tempo medido.

4.6 Tabelas completas

As Tabelas 4.1, 4.2 e 4.3 reúnem todas as métricas de todas as configurações, uma por benchmark, com a explicação das colunas logo abaixo. Elas são geradas dos mesmos arquivos CSV que originam as figuras.

TABELA 4.1 – Métricas por condição e modelo no benchmark BIPIA, médias de 10 repetições.

model	condition	n	asr	utility	overBlock	invalidOutput	markedRate	meanTokens	latP50
llama-3.1-8b	simple	300	0.123	0.564	0.021	0.001	0.000	1067.204	1.839
llama-3.1-8b	delimiting	300	0.103	0.539	0.067	0.001	0.000	1109.227	1.820
llama-3.1-8b	demarking	300	0.047	0.493	0.027	0.001	0.069	1203.440	1.814
llama-3.1-8b	delimiting_demarking	300	0.039	0.482	0.065	0.001	0.079	1232.852	1.762
gpt-oss-20b	simple	300	0.235	0.661	0.001	0.074	0.000	1411.183	4.872
gpt-oss-20b	delimiting	300	0.129	0.661	0.000	0.030	0.000	1408.372	4.962
gpt-oss-20b	demarking	300	0.153	0.639	0.001	0.069	0.041	1514.817	4.837
gpt-oss-20b	delimiting_demarking	300	0.097	0.638	0.000	0.037	0.021	1505.214	4.823
llama-3.1-8b	no_filter	300	0.184	0.587	0.000	0.000	0.000	527.205	1.024
gpt-oss-20b	no_filter	300	0.795	0.660	0.000	0.000	0.000	794.128	2.932
gpt-oss-safeguard	simple	300	0.374	0.661	0.000	0.000	0.000	1492.515	2.497
gpt-oss-safeguard	delimiting	300	0.314	0.654	0.000	0.000	0.000	1558.584	2.420
gpt-oss-safeguard	demarking	300	0.305	0.649	0.000	0.001	0.030	1611.346	2.377
gpt-oss-safeguard	delimiting_demarking	300	0.235	0.646	0.000	0.000	0.033	1657.387	2.104

Legenda. model: modelo que instancia o sistema (filtro e componente principal); nas linhas de guard, o modelo do filtro; **condition:** condição experimental: simple, delimiting, demarking, delimiting_demarking, ou a referência no_filter; **n:** número de itens efetivamente executados na configuração; **asr:** taxa de sucesso do ataque na saída do componente principal, sobre os itens de ataque (métrica primária; menor é melhor); **utility:** fração das tarefas benignas cumpridas; item bloqueado conta como fracasso (maior é melhor); **overBlock:** fração dos itens benignos que o filtro bloqueou (sobre-bloqueio); **invalidOutput:** fração das saídas do filtro que não puderam ser interpretadas; contam conservadoramente como não bloqueio; **markedRate:** fração dos itens liberados que receberam ao menos um trecho marcado, que é o alcance efetivo da marcação seletiva; **meanTokens:** média de tokens (entrada + saída) por item, somando os dois estágios; **latP50:** tempo de geração mediano por item, em segundos, medido pelo provedor e somando os dois estágios; exclui fila e rede.

TABELA 4.2 – Métricas por condição e modelo no benchmark SEP, médias de 10 repetições.

model	condition	n	asr	utility	overBlock	invalidOutput	markedRate	meanTokens	latP50
llama-3.1-8b	simple	300	0.097	0.967	0.033	0.000	0.000	792.746	3.700
llama-3.1-8b	delimiting	300	0.087	0.949	0.051	0.000	0.000	867.524	3.682
llama-3.1-8b	demarking	300	0.075	0.966	0.034	0.000	0.022	879.112	2.837
llama-3.1-8b	delimiting_demarking	300	0.069	0.954	0.046	0.000	0.112	948.382	2.929
gpt-oss-20b	simple	300	0.743	0.977	0.001	0.013	0.000	1733.365	9.577
gpt-oss-20b	delimiting	300	0.735	0.979	0.003	0.010	0.000	1773.805	10.151
gpt-oss-20b	demarking	300	0.691	0.993	0.001	0.014	0.004	1632.028	8.431
gpt-oss-20b	delimiting_demarking	300	0.657	0.993	0.002	0.013	0.006	1719.191	8.935
llama-3.1-8b	no_filter	300	0.101	1.000	0.000	0.000	0.000	484.226	2.887
gpt-oss-20b	no_filter	300	0.783	0.984	0.000	0.000	0.000	1241.679	7.312
gpt-oss-safeguard	simple	300	0.779	0.981	0.000	0.000	0.000	1705.883	7.411
gpt-oss-safeguard	delimiting	300	0.785	0.978	0.001	0.000	0.000	1852.056	7.664
gpt-oss-safeguard	demarking	300	0.701	0.990	0.001	0.000	0.001	1663.635	6.664
gpt-oss-safeguard	delimiting_demarking	300	0.692	0.995	0.000	0.000	0.002	1765.446	6.503

Legenda. model: modelo que instancia o sistema (filtro e componente principal); nas linhas de guard, o modelo do filtro; **condition:** condição experimental: simple, delimiting, demarking, delimiting_demarking, ou a referência no_filter; **n:** número de itens efetivamente executados na configuração; **asr:** taxa de sucesso do ataque na saída do componente principal, sobre os itens de ataque (métrica primária; menor é melhor); **utility:** fração das tarefas benignas cumpridas; item bloqueado conta como fracasso (maior é melhor); **overBlock:** fração dos itens benignos que o filtro bloqueou (sobre-bloqueio); **invalidOutput:** fração das saídas do filtro que não puderam ser interpretadas; contam conservadoramente como não bloqueio; **markedRate:** fração dos itens liberados que receberam ao menos um trecho marcado, que é o alcance efetivo da marcação seletiva; **meanTokens:** média de tokens (entrada + saída) por item, somando os dois estágios; **latP50:** tempo de geração mediano por item, em segundos, medido pelo provedor e somando os dois estágios; exclui fila e rede.

TABELA 4.3 – Métricas por condição e modelo no benchmark NotInject, médias de 10 repetições.

model	condition	n	asr	utility	overBlock	invalidOutput	markedRate	meanTokens	latP50
llama-3.1-8b	simple	339	–	0.902	0.098	0.000	0.000	772.803	4.460
llama-3.1-8b	delimiting	339	–	0.919	0.081	0.001	0.000	873.287	4.628
llama-3.1-8b	demarking	339	–	0.901	0.099	0.000	0.002	830.562	3.490
llama-3.1-8b	delimiting_demarking	339	–	0.921	0.079	0.001	0.002	910.876	3.689
gpt-oss-20b	simple	339	–	0.922	0.044	0.017	0.000	1881.129	10.591
gpt-oss-20b	delimiting	339	–	0.906	0.055	0.011	0.000	1953.257	10.559
gpt-oss-20b	demarking	339	–	0.932	0.047	0.013	0.001	1635.738	8.136
gpt-oss-20b	delimiting_demarking	339	–	0.932	0.054	0.013	0.003	1661.217	8.143
llama-3.1-8b	no_filter	339	–	1.000	0.000	0.000	0.000	540.755	3.879
gpt-oss-20b	no_filter	339	–	0.964	0.000	0.000	0.000	1477.801	8.057
gpt-oss-safeguard	simple	339	–	0.948	0.016	0.000	0.000	1879.547	8.492
gpt-oss-safeguard	delimiting	339	–	0.942	0.019	0.000	0.000	1993.751	8.432
gpt-oss-safeguard	demarking	339	–	0.969	0.015	0.000	0.000	1626.800	6.008
gpt-oss-safeguard	delimiting_demarking	339	–	0.962	0.019	0.000	0.001	1693.818	6.032

Legenda. **model:** modelo que instancia o sistema (filtro e componente principal); nas linhas de guard, o modelo do filtro; **condition:** condição experimental: simple, delimiting, demarking, delimiting_demarking, ou a referência no_filter; **n:** número de itens efetivamente executados na configuração; **asr:** taxa de sucesso do ataque na saída do componente principal, sobre os itens de ataque (métrica primária; menor é melhor); **utility:** fração das tarefas benignas cumpridas; item bloqueado conta como fracasso (maior é melhor); **overBlock:** fração dos itens benignos que o filtro bloqueou (sobre-bloqueio); **invalidOutput:** fração das saídas do filtro que não puderam ser interpretadas; contam conservadoramente como não bloqueio; **markedRate:** fração dos itens liberados que receberam ao menos um trecho marcado, que é o alcance efetivo da marcação seletiva; **meanTokens:** média de tokens (entrada + saída) por item, somando os dois estágios; **latP50:** tempo de geração mediano por item, em segundos, medido pelo provedor e somando os dois estágios; exclui fila e rede.

5 Conclusão

A pergunta que estruturou este trabalho pedia uma grandeza, não uma opinião: qual o ganho, medido na saída do sistema, de implementar separação entre canal de instrução e canal de dado em um *guardrail* de entrada. As 420 execuções descritas no Capítulo 3, dez repetições da mesma bateria, permitem responder com número e intervalo, e é o que esta seção faz, na ordem em que as medidas foram apresentadas no Capítulo 4. Todo número citado é a média das dez repetições.

5.1 O ganho da separação de canal

No BIPIA, único dos três *benchmarks* que publica resposta de referência e portanto o único em que ataque e utilidade são julgados com o mesmo rigor, aplicar os dois fatores simultaneamente reduziu a taxa de sucesso do ataque em 0,084 no Llama-3.1-8B (IC 95% de $-0,122$ a $-0,055$) e em 0,139 no gpt-oss-20b (IC 95% de $-0,185$ a $-0,099$). A comparação é sempre contra o *prompt* simples, isto é, contra o mesmo filtro, sobre os mesmos itens, sem separação alguma. Os dois intervalos ficam inteiramente abaixo do zero.

Cada fator, aplicado sozinho, também reduziu o ataque nos dois modelos, mas o peso de cada um muda com o modelo. No Llama, a marcação seletiva responde por quase todo o ganho: $-0,075$ (IC de $-0,111$ a $-0,047$), contra $-0,019$ da delimitação (IC de $-0,041$ a $-0,003$). No gpt-oss, a delimitação é a mais forte, com $-0,107$ (IC de $-0,154$ a $-0,065$), e a marcação vem logo atrás, com $-0,082$ (IC de $-0,113$ a $-0,055$).

Os efeitos principais do desenho fatorial, que usam as quatro condições de uma vez e por isso medem cada fator com mais precisão, confirmam a leitura e acrescentam uma diferença entre os modelos. No Llama, o efeito da marcação é de $-0,070$ (IC de $-0,101$ a $-0,046$), o da delimitação é pequeno, de $-0,014$ (IC de $-0,029$ a $-0,002$), e a interação é compatível com zero ($+0,011$, IC de $-0,011$ a $+0,033$): os dois fatores somam. No gpt-oss, os dois efeitos principais são claros, $-0,082$ para a delimitação (IC de $-0,119$ a $-0,050$) e $-0,057$ para a marcação (IC de $-0,077$ a $-0,040$), e a interação é positiva, de $+0,050$ (IC de $+0,013$ a $+0,087$): com a delimitação já presente, a marcação acrescenta

menos do que acrescenta sozinha. A leitura mais econômica é que os dois pontos de intervenção corrigem falhas em parte diferentes: a delimitação organiza o que o filtro lê, a marcação transporta adiante o que ele descobriu. No modelo menor as duas falhas são independentes e os ganhos se somam; no maior, parte dos ataques que a delimitação passa a detectar é a mesma que a marcação neutralizaria, e a combinação rende menos que a soma, embora continue sendo a melhor das quatro condições.

No SEP, cuja tarefa é justamente a de separar instrução de dado, os efeitos são menores, e quem os carrega é a marcação. O efeito principal da marcação é negativo e distinto de zero nas três linhas: $-0,020$ no Llama (IC de $-0,043$ a $-0,003$), $-0,065$ no gpt-oss (IC de $-0,103$ a $-0,032$) e $-0,086$ no filtro treinado (IC de $-0,130$ a $-0,051$). O da delimitação é nulo no Llama e no filtro treinado, e no gpt-oss tem o limite superior encostado no zero. A combinação reduziu o ataque em $0,029$ no Llama (IC de $-0,055$ a $-0,009$) e em $0,086$ no gpt-oss (IC de $-0,127$ a $-0,049$). O contraste com o BIPIA é coerente com a natureza dos dois conjuntos: a sonda do SEP é uma instrução plausível embutida em dado legítimo, sem a assinatura léxica que torna um ataque do BIPIA reconhecível, e um filtro que decide por leitura tem menos de que se agarrar para bloquear. Como o filtro raramente aponta trecho no SEP (Seção 5.4), o ganho ali vem sobretudo do aviso de cautela que a condição acrescenta ao componente principal, e não das marcas.

5.2 A dimensão do efeito

Medido contra a referência sem filtro, o efeito da separação de canal é uma fração da proteção total, e essa fração depende do modelo. No gpt-oss-20b, o *guardrail* clássico já derruba a taxa de sucesso do ataque em $0,560$ no BIPIA, e a separação de canal acrescenta a isso os $0,139$ da Seção 5.1, levando a redução total a $0,699$. No Llama-3.1-8B a proporção é outra e mais favorável à tese: o filtro sem separação reduz $0,061$, e a versão com os dois fatores reduz $0,145$, ou seja, mais que o dobro.

Esse par de números delimita bem o alcance da contribuição. Ter um filtro continua sendo a decisão de maior impacto. Separar os canais dentro do filtro é um refinamento barato que, no modelo menor, chega a duplicar a proteção que o filtro já oferecia.

5.3 O custo

Uma defesa que suprime o ataque às custas da tarefa legítima não serve, e aqui os dois modelos se separam pela magnitude. No gpt-oss-20b o custo de utilidade no BIPIA é pequeno e vem só da marcação: a delimitação não custou nada ($+0,001$, IC de $-0,005$ a $+0,008$), e a combinação custou $0,023$ (IC de $-0,050$ a $-0,007$), o mesmo que a mar-

cação sozinha. No Llama-3.1-8B o custo é maior e vem dos dois fatores, por mecanismos diferentes. A combinação custou 0,082 (IC de $-0,123$ a $-0,044$), o efeito principal da marcação é de $-0,064$ (IC de $-0,101$ a $-0,032$) e o da delimitação, de $-0,018$ (IC de $-0,039$ a $-0,002$).

O custo da delimitação no Llama é de bloqueio: o sobre-bloqueio no BIPIA sobe de 0,021 sob o *prompt* simples para 0,067 com delimitação, enquanto a fração de tarefas cumpridas entre os itens liberados não muda (0,576 e 0,578). O custo da marcação não vem das marcas nem do bloqueio. Os itens benignos que receberam marca tiveram a tarefa cumprida em 0,864 das vezes sob marcação, acima da média; os liberados sem marca alguma, que são a grande maioria, caíram de 0,576 para 0,485. O que muda para eles é o aviso de cautela que a marcação acrescenta ao *prompt* do componente principal, e o Apêndice A mostra o Llama recusando uma pergunta legítima, sobre um texto sem marca, depois de ler esse aviso. O gpt-oss-20b recebe o mesmo aviso e paga o mesmo tipo de custo, em escala muito menor (0,661 para 0,640). O aviso é também parte da proteção, como mostra a Seção 5.4: o mesmo texto que torna o componente principal mais desconfiado diante do ataque o torna mais relutante diante da tarefa legítima.

No NotInject, o conjunto construído para expor o sobre-bloqueio, nenhuma condição se afastou muito do filtro sem separação: o Llama ficou entre 0,079 e 0,099, com a delimitação reduzindo o sobre-bloqueio em vez de aumentá-lo, e o gpt-oss entre 0,044 e 0,055.

A despesa aparece em *tokens*, e é pequena diante da que já se paga por ter um filtro. As duas intervenções acrescentam texto ao *prompt*, mas o filtro em si já duplica o número de chamadas ao modelo. O tempo de geração segue a mesma proporção, com a ressalva de que o provedor que atende cada chamada varia, e com ele o hardware.

5.4 O teto do método

A marcação seletiva só age sobre trechos que o filtro efetivamente apontou. Quando ele não aponta nada, não há o que marcar, e a condição degenera no *prompt* simples. O alcance medido é modesto: no BIPIA, 0,069 dos itens liberados receberam marca sob marcação isolada no Llama e 0,079 sob a combinação, contra 0,041 e 0,021 no gpt-oss. No SEP, a combinação no Llama chegou a 0,112 e a marcação isolada a 0,022; nas linhas do gpt-oss, o alcance ficou abaixo de 0,01.

Onde há marca, ela funciona. No BIPIA, entre os ataques liberados com marca, a execução foi de 0,037 no Llama e nula no gpt-oss sob marcação, contra 0,205 e 0,738 entre os ataques liberados sob o *prompt* simples. O fator medido como marcação, porém, inclui também o aviso de cautela no *prompt* do componente principal, presente em toda requisição da condição, com ou sem marca, e o aviso sozinho já muda o comportamento:

entre os ataques liberados sem marca alguma, a execução cai de 0,205 para 0,084 no Llama e de 0,738 para 0,617 no gpt-oss. Parte do ganho atribuído à marcação vem, portanto, do aviso, e não das marcas, e o mesmo aviso é a fonte do custo de utilidade descrito na Seção 5.3. O desenho não separa as duas partes, porque o aviso só aparece junto com a marcação.

O que limita a parte da marca não é a ideia de preservar e sinalizar, e sim a capacidade do filtro de localizar o trecho a sinalizar. Essa é uma limitação de componente, e não de arquitetura: um filtro que aponte melhor aproveita a mesma cadeia sem que nada mais precise mudar.

5.5 Comparação com o filtro treinado

O desenho incluiu o gpt-oss-safeguard, ajustado por *fine-tuning* sobre o mesmo *backbone* do gpt-oss-20b, no papel de filtro e sob as mesmas quatro condições. Isso transforma em medição direta o que de outra forma seria citação de número alheio, e permite duas leituras: entre as linhas, o que o treinamento acrescenta; dentro da linha treinada, o que a separação de canal acrescenta a um filtro que já foi ajustado.

A segunda leitura é a que responde à pergunta de pesquisa, e responde afirmativamente. Sobre o filtro treinado, a taxa de sucesso do ataque no BIPIA cai de 0,374 sob o *prompt* simples para 0,235 com delimitação e marcação, uma redução de 0,139 (IC de $-0,181$ a $-0,106$), com as condições intermediárias em 0,314 e 0,305. A redução absoluta é a mesma do modelo aberto da família. A detecção acompanha: a taxa de detecção a 1% de falsos positivos sobe de 0,568 para 0,675. No SEP o ataque cai de 0,779 para 0,692, inteiramente pela marcação, e a utilidade não cai de forma relevante em nenhum dos dois: no BIPIA a combinação custou 0,015 (IC de $-0,040$ a $-0,001$), e no SEP a utilidade subiu. O efeito da configuração, portanto, não depende de o filtro ser um modelo de uso geral.

A primeira leitura contraria a expectativa de que treinar seja o caminho mais curto. Em todas as quatro condições o filtro treinado protegeu menos que o modelo aberto da mesma família: 0,374 contra 0,235 sob o *prompt* simples, e 0,235 contra 0,097 na melhor configuração. A melhor configuração do modelo treinado apenas empata com o *prompt* simples do modelo aberto.

O resultado pede explicação, e os registros a fornecem. Os dados estão íntegros: a saída do filtro treinado foi válida em mais de 99,9% das decisões, e, entre os ataques que ele deixou passar sob o *prompt* simples, a fração executada pelo componente principal foi a mesma observada sem filtro (0,796 e 0,795). Toda a diferença está na detecção. E a detecção falha por um motivo de tarefa, não de capacidade: o modelo foi ajustado para

classificar conteúdo nocivo segundo a política escrita no *prompt* de sistema, e a política recebida foram as mesmas linhas entregues a todas as condições. Um ataque do tipo “acrescente um link” não viola política de dano alguma, e o modelo o deixou passar. O SEP confirma a leitura: a sonda daquele *benchmark* é uma instrução inofensiva embutida no dado, e o filtro treinado bloqueou 0 das 1.500 decisões sobre itens atacados sob o *prompt* simples e 2 com delimitação e marcação, enquanto o modelo aberto bloqueou 28 e 78. O NotInject mostra o outro lado da mesma moeda: itens benignos com vocabulário sensível também não são conteúdo nocivo, e ali o filtro treinado sobre-bloqueou menos (0,016 contra 0,044 sob o *prompt* simples). Ele erra menos contra quem é inocente e detecta menos quem ataca, que é exatamente o perfil de um classificador de dano posto a procurar injeção.

A leitura correta do conjunto é, portanto, mais estreita do que a frase “treinar não bastou” sugere: um modelo geral com um *prompt* dirigido à injeção superou, nesse papel, o classificador de dano da própria família, e a separação de canal melhorou os dois. Isso não estabelece um teto para o que um *fine-tuning* dirigido à injeção alcançaria, como o DataFilter (WANG *et al.*, 2025) ou o PromptShield (JACOB *et al.*, 2025); esses não foram medidos, pelo motivo exposto na Seção 5.6.

5.6 Limitações

Quatro limitações restringem o alcance do que foi medido, e nenhuma delas foi descoberta depois do fato: todas são propriedades conhecidas do protocolo.

A primeira é o julgamento do ataque. O casamento com a testemunha não distingue obedecer de citar: um modelo que reproduza o conteúdo externo ao resumi-lo pode fazer a testemunha aparecer sem jamais ter seguido a instrução. O viés é sistemático e empurra a favor da hipótese nula, o que torna as reduções relatadas conservadoras.

A segunda é o significado de utilidade fora do BIPIA. SEP e NotInject não publicam resposta de referência, e ali a métrica exprime “respondeu sem ter sido bloqueado”, não “respondeu corretamente”. Comparações de utilidade entre *benchmarks* não são legítimas, e o texto evitou fazê-las.

A terceira é a instabilidade da decisão individual. Os números deste trabalho são médias de dez repetições, e os intervalos levam em conta a variação entre elas; a média, porém, esconde que a decisão sobre um mesmo item muda de uma execução para outra. No BIPIA, sob o *prompt* simples, o filtro não tomou a mesma decisão nas dez repetições em 31% dos itens no Llama e em 35% no gpt-oss, contra 12% no filtro treinado, que decide de forma mais estável, ainda que decida pior nesta tarefa. Um sistema em operação executa cada requisição uma vez, e o que ele entrega a um usuário é uma amostra dessa

variação, não a média. A taxa de saída inválida do filtro, que chegou a 0,074 no gpt-oss e é contabilizada de forma conservadora como não bloqueio, pertence à mesma família de ressalvas.

A quarta é o alcance da comparação com modelos treinados. O filtro fine-tunado avaliado foi usado fora da tarefa para a qual foi ajustado, como discutido na Seção 5.5: ele classifica conteúdo nocivo, e a comparação mede o que a família oferece pronto, não o limite do *fine-tuning* para injeção. Nenhuma defesa por remoção foi medida, e a razão é de disponibilidade: os detectores treinados especificamente para *prompt injection* sobre o *backbone* avaliado (WANG *et al.*, 2025; GENG *et al.*, 2025; JACOB *et al.*, 2025) existem como pesos publicados, mas não são servidos por interface de API, e hospedá-los sairia do custo e da reprodutibilidade adotados. A comparação entre marcar e remover permanece, assim, argumentativa e apoiada na literatura, não medida nesta bateria.

Referências

- BEURER-KELLNER, L.; BUESSER, B.; CREȚU, A.-M.; DEBENEDETTI, E.; DOBOS, D.; FABIAN, D.; FISCHER, M.; FROELICHER, D.; GROSSE, K.; NAEFF, D.; OZOANI, E.; PAVERD, A.; TRAMÈR, F.; VOLHEJN, V. Design patterns for securing LLM agents against prompt injections. **arXiv preprint arXiv:2506.08837**, 2025.
- CHEN, S.; PIET, J.; SITAWARIN, C.; WAGNER, D. StruQ: Defending against prompt injection with structured queries. *In: 34th USENIX Security Symposium. Proceedings [...]. [S.l.: s.n.], 2025.*
- CHEN, S.; ZHARMAGAMBETOV, A.; MAHLOUJIFAR, S.; CHAUDHURI, K.; WAGNER, D.; GUO, C. SecAlign: Defending against prompt injection with preference optimization. *In: ACM SIGSAC Conference on Computer and Communications Security (CCS). Proceedings [...]. [S.l.: s.n.], 2025.*
- CHEN, S.; ZHARMAGAMBETOV, A.; WAGNER, D.; GUO, C. Meta SecAlign: A secure foundation LLM against prompt injection attacks. **arXiv preprint arXiv:2507.02735**, 2025.
- CHENNABASAPPA, S.; NIKOLAIDIS, C.; SONG, D.; MOLNAR, D.; DING, S.; WAN, S.; WHITMAN, S.; DEASON, L.; DOUCETTE, N.; MONTILLA, A. *et al.* LlamaFirewall: An open source guardrail system for building secure AI agents. **arXiv preprint arXiv:2505.03574**, 2025.
- DEBENEDETTI, E.; SHUMAILOV, I.; FAN, T.; HAYES, J.; CARLINI, N.; FABIAN, D.; KERN, C.; SHI, C.; TERZIS, A.; TRAMÈR, F. Defeating prompt injections by design. **arXiv preprint arXiv:2503.18813**, 2025.
- DEBENEDETTI, E.; ZHANG, J.; BALUNOVIĆ, M.; BEURER-KELLNER, L.; FISCHER, M.; TRAMÈR, F. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *In: Advances in Neural Information Processing Systems, Datasets and Benchmarks Track. Proceedings [...]. [S.l.: s.n.], 2024.*
- DONG, Y.; MU, R.; JIN, G.; QI, Y.; HU, J.; ZHAO, X.; MENG, J.; RUAN, W.; HUANG, X. Position: Building guardrails for large language models requires systematic design. *In: Proceedings of the 41st International Conference on Machine Learning (ICML). Proceedings [...]. [S.l.: s.n.], 2024.*

EFRON, B. Better bootstrap confidence intervals. **Journal of the American Statistical Association**, v. 82, n. 397, p. 171–185, 1987.

GENG, R.; WANG, Y.; YIN, C.; CHENG, M.; CHEN, Y.; JIA, J. PISanitizer: Preventing prompt injection to long-context LLMs via prompt sanitization. **arXiv preprint arXiv:2511.10720**, 2025.

GRESHAKE, K.; ABDELNABI, S.; MISHRA, S.; ENDRES, C.; HOLZ, T.; FRITZ, M. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security. Proceedings [...]. [S.l.: s.n.]*, 2023. p. 79–90.

HACKETT, W.; BIRCH, L.; TRAWICKI, S.; SURI, N.; GARRAGHAN, P. Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. **arXiv preprint arXiv:2504.11168**, 2025.

HINES, K.; LOPEZ, G.; HALL, M.; ZARFATI, F.; ZUNGER, Y.; KICIMAN, E. Defending against indirect prompt injection attacks with spotlighting. *In: Conference on Applied Machine Learning for Information Security (CAMLIS). Proceedings [...]. [S.l.: s.n.]*, 2024.

INAN, H.; UPASANI, K.; CHI, J.; RUNGTA, R.; IYER, K.; MAO, Y.; TONTCHEV, M.; HU, Q.; FULLER, B.; TESTUGGINE, D.; KHABSA, M. Llama guard: LLM-based input-output safeguard for human-ai conversations. **arXiv preprint arXiv:2312.06674**, 2023.

JACOB, D.; ALZHRANI, H.; HU, Z.; ALOMAIR, B.; WAGNER, D. PromptShield: Deployable detection for prompt injection attacks. *In: Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy (CODASPY). Proceedings [...]. [S.l.: s.n.]*, 2025.

LEWIS, P.; PEREZ, E.; PIKTUS, A.; PETRONI, F.; KARPUKHIN, V.; GOYAL, N.; KÜTTLER, H.; LEWIS, M.; YIH, W. tau; ROCKTÄSCHEL, T.; RIEDEL, S.; KIELA, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. *In: Advances in Neural Information Processing Systems 33. Proceedings [...]. [S.l.: s.n.]*, 2020.

LI, H.; LIU, X.; XIAO, C. InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models. **arXiv preprint arXiv:2410.22770**, 2024.

LIU, Y.; JIA, Y.; GENG, R.; JIA, J.; GONG, N. Z. Formalizing and benchmarking prompt injection attacks and defenses. *In: 33rd USENIX Security Symposium. Proceedings [...]. [S.l.: s.n.]*, 2024.

OUYANG, L.; WU, J.; JIANG, X.; ALMEIDA, D.; WAINWRIGHT, C. L.; MISHKIN, P.; ZHANG, C.; AGARWAL, S.; SLAMA, K.; RAY, A. *et al.* Training language models to follow instructions with human feedback. *In: Advances in Neural Information Processing Systems 35. Proceedings [...]. [S.l.: s.n.]*, 2022.

PEREZ, F.; RIBEIRO, I. Ignore previous prompt: Attack techniques for language models. *In: NeurIPS Workshop on ML Safety. Proceedings [...]. [S.l.: s.n.]*, 2022.

- SHI, T.; ZHU, K.; WANG, Z.; JIA, Y.; CAI, W.; LIANG, W.; WANG, H.; ALZAHIRANI, H.; LU, J.; KAWAGUCHI, K.; ALOMAIR, B.; ZHAO, X.; WANG, W. Y.; GONG, N.; GUO, W.; SONG, D. PromptArmor: Simple yet effective prompt injection defenses. **arXiv preprint arXiv:2507.15219**, 2025.
- VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. *In: Advances in Neural Information Processing Systems 30. Proceedings [...]. [S.l.: s.n.], 2017.*
- WALLACE, E.; XIAO, K.; LEIKE, R.; WENG, L.; HEIDECHE, J.; BEUTEL, A. The instruction hierarchy: Training LLMs to prioritize privileged instructions. **arXiv preprint arXiv:2404.13208**, 2024.
- WANG, Y.; CHEN, S.; ALKHUHAIR, R.; ALOMAIR, B.; WAGNER, D. Defending against prompt injection with DataFilter. **arXiv preprint arXiv:2510.19207**, 2025.
- WEBER, I. Large language models as software components: A taxonomy for LLM-integrated applications. **arXiv preprint arXiv:2406.10300**, 2024.
- XIE, Y.; YI, J.; SHAO, J.; CURL, J.; LYU, L.; CHEN, Q.; XIE, X.; WU, F. Defending ChatGPT against jailbreak attack via self-reminders. **Nature Machine Intelligence**, v. 5, n. 12, p. 1486–1496, 2023.
- YI, J.; XIE, Y.; ZHU, B.; KICIMAN, E.; SUN, G.; XIE, X.; WU, F. Benchmarking and defending against indirect prompt injection attacks on large language models. *In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining. Proceedings [...]. [S.l.: s.n.], 2025.*
- ZENG, W.; LIU, Y.; MULLINS, R.; PERAN, L.; FERNANDEZ, J.; HARKOUS, H.; NARASIMHAN, K.; PROUD, D.; KUMAR, P.; RADHARAPU, B.; STURMAN, O.; WAHLTINEZ, O. ShieldGemma: Generative AI content moderation based on Gemma. **arXiv preprint arXiv:2407.21772**, 2024.
- ZHAO, H.; YUAN, C.; HUANG, F. *et al.* Qwen3Guard technical report. **arXiv preprint arXiv:2510.14276**, 2025.
- ZVEREV, E.; ABDELNABI, S.; TABESH, S.; FRITZ, M.; LAMPERT, C. H. Can LLMs separate instructions from data? and what do we even mean by that? *In: International Conference on Learning Representations (ICLR). Proceedings [...]. [S.l.: s.n.], 2025.*
- ZVEREV, E.; KORTUKOV, E.; PANFILOV, A.; VOLKOVA, A.; TABESH, S.; LAPUSCHKIN, S.; SAMEK, W.; LAMPERT, C. H. ASIDE: Architectural separation of instructions and data in language models. *In: International Conference on Learning Representations (ICLR). Proceedings [...]. [S.l.: s.n.], 2026.*